cnavier

Numerical Methodology and Scientific User Manual

cnavier project

Repository state: `main` at `5018e29bff534b252a7ac9025ece4ac43efd78ad`
6 October 2026

Abstract

`cnavier` is a compact two-dimensional incompressible Navier–Stokes solver written in C, with optional OpenMP and CUDA acceleration. The code advances the scalar vorticity equation and reconstructs a divergence-free velocity field from a stream function. Spatial derivatives are represented by finite-difference matrices of selectable interior order (second, fourth, or sixth), assembled into two-dimensional sparse operators through Kronecker products and applied in compressed sparse row (CSR) form. Time integration is explicit, using either forward Euler or classical fourth-order Runge–Kutta (RK4). The stream-function Poisson equation may be solved by Gauss–Seidel, successive over-relaxation (SOR), or a direct discrete-sine-transform solver implemented with FFTW3 on the CPU and cuFFT on the GPU.

This document describes the mathematical model, the exact discretizations present in the source code, solver sequencing, boundary treatment, stability considerations, verification and validation strategy, parallel backends, usage, and current numerical limitations. It is intended to be both a scientific description of the methodology and a practical reference for users and developers.

Contents

1 Introduction **4**
1.1 Repository organization 4

2 Governing Equations **5**
2.1 Nondimensional incompressible Navier–Stokes equations 5
2.2 Vorticity–streamfunction formulation 5
2.3 Why incompressibility is naturally enforced 6

3 Computational Grid and Data Layout **6**
3.1 Domain and spacing 6
3.2 Grid convention 6

4 Finite-Difference Spatial Discretization **7**
4.1 First derivative 7
4.2 Second derivative 7
4.3 Boundary closure and formal order 7
4.4 Two-dimensional operators by Kronecker product 8

5 Boundary Conditions and Wall Vorticity **8**
5.1 Velocity boundary conditions 8
5.2 Vorticity boundary treatment 8
5.3 Streamfunction boundary condition 9

6 Vorticity Transport Discretization **9**

7 Time Integration **10**
7.1 Forward Euler . . . 10
7.2 Classical fourth-order Runge–Kutta . . . 10

8 Poisson Equation for the Streamfunction **11**
8.1 Five-point discrete Poisson operator . . . 11
8.2 Gauss–Seidel . . . 11
8.3 Successive over-relaxation . . . 11
8.4 Direct FFT/DST-I solver . . . 12
8.5 Forcing and drag . . . 13
8.6 Fourth-order compact Poisson operator . . . 14
8.7 Doubly periodic boundaries . . . 14
8.8 Compact schemes on the periodic grid . . . 15
8.9 Operators in Fourier space and pseudospectral differentiation . . . 15

9 Complete Timestep Algorithms **17**
9.1 Euler sequence . . . 17
9.2 RK4 sequence . . . 17
9.3 Pseudocode . . . 18

10 Stability and Timestep Selection **18**
10.1 Implemented Courant check . . . 18
10.2 Implemented viscous limit . . . 19
10.3 Diffusive scaling . . . 19
10.4 Centered advection . . . 19
10.5 Recommended practice . . . 20

11 Sparse Linear Algebra **20**

12 Parallel Implementations **20**
12.1 Backend structure . . . 20
12.2 OpenMP . . . 20
12.3 CUDA . . . 21
12.4 GPU sine transform through odd extension . . . 22

13 Diagnostics and Output **22**
13.1 Discrete continuity diagnostic . . . 22
13.2 VTK output . . . 23
13.3 Centerline profiles . . . 23
13.4 Flow diagnostics . . . 23

14 Verification and Validation **24**
14.1 Analytic verification of the FFT Poisson solver . . . 24
14.2 Residual verification of the iterative Poisson solvers . . . 24
14.3 Short cavity integration tests . . . 25
14.4 CPU–GPU equivalence tests . . . 25
14.5 Spatial convergence against a manufactured solution . . . 25
14.6 What limits the spatial order: an ablation . . . 26
14.7 Raising the order with walls . . . 27
14.8 Spatial order without walls . . . 28
14.9 Diagnostics . . . 29

14.10Forcing and drag . . . 29
14.11Two-dimensional turbulence . . . 30
14.12Physical validation against Ghia et al. . . . 38

15 Performance Characteristics **38**

16 Building and Running **39**
16.1 Dependencies . . . 39
16.2 Build modes . . . 40
16.3 Tests . . . 40
16.4 Runtime options . . . 41

17 Default Numerical Configuration **43**

18 Numerical Interpretation and Current Limitations **43**
18.1 Mixed formal spatial order . . . 43
18.2 Poisson operator differs from transport Laplacian . . . 44
18.3 Boundary vorticity accuracy . . . 44
18.4 Explicit stability . . . 44
18.5 Steady-state convergence . . . 44
18.6 Pressure is not solved . . . 44
18.7 Geometry and dimensionality . . . 44

19 Suggested Scientific Workflow **44**

20 Possible Methodological Extensions **45**

21 Polar Grid: Flow Past a Circular Cylinder **45**
21.1 Conformal coordinates . . . 46
21.2 Poisson solver . . . 46
21.3 Boundary conditions . . . 46
21.4 Time step and diagnostics . . . 47
21.5 Running the case . . . 47
21.6 Verification and validation . . . 47

22 Final Remarks **49**

A Mapping Between Equations and Source Code **49**

B Reproducibility Record **50**

1 Introduction

The objective of `cnavier` is to provide a small, inspectable computational fluid dynamics (CFD) code for two-dimensional incompressible viscous flow. Its reference problem is the square lid-driven cavity: three stationary no-slip walls and one tangentially moving wall. Despite the small code base, the project contains several numerical alternatives that are useful for study and benchmarking: multiple finite-difference stencil widths, two explicit time integrators, three elliptic solvers, a sparse-operator formulation, shared-memory parallelism, and an optional CUDA implementation of the complete time loop.

The code uses a *vorticity–streamfunction* formulation rather than the primitive-variable (u, v, p) equations. This choice removes pressure from the evolution system and replaces the pressure–velocity coupling problem by a scalar Poisson equation. In two dimensions, the streamfunction definition also gives a particularly strong discrete incompressibility property: when the discrete x- and y-derivative operators commute, the recovered velocity has zero discrete divergence to round-off.

The solver is designed primarily as an educational and experimental code rather than as a general-purpose production CFD package. The current implementation assumes a rectangular Cartesian domain, a Cartesian computational array, constant Reynolds number, explicit time stepping, and Dirichlet wall velocities. These constraints simplify the formulation sufficiently that each numerical operation can be followed directly from the governing equations to its C or CUDA implementation.

1.1 Repository organization

The main numerical components are:

- `src/main.c`: run configuration, command line, operator construction, time loop, diagnostics, and output;
- `src/backend.c`: one interface over the CPU and CUDA solvers, and the choice between them;
- `src/finitediff.c`: one-dimensional first- and second-derivative finite-difference matrices, with walls or periodic, and CSR conversion;
- `src/linearalg.c`: dense storage utilities, CSR sparse matrices, sparse matrix–vector multiplication, and Kronecker products;
- `src/fluiddyn.c`: wall conditions, vorticity transport, Euler and RK4 integration, recovery of the velocity from the streamfunction, and full timesteps;
- `src/poisson.c`: Gauss–Seidel, SOR, and FFTW/DST-I Poisson solvers, and the periodic FFT solver;
- `src/cudasolver.cu`: CUDA implementations of the time loop, sparse operators, elliptic solvers, and diagnostics;
- `src/threads.c`: the default number of OpenMP threads (one per physical core);
- `src/forcing.c`: Kolmogorov and random forcing;
- `src/diagnostics.c`: energy, enstrophy and palinstrophy, and on periodic grids spectra and spectral fluxes;
- `src/utils.c`: VTK output, centerline data for comparison with Ghia et al., and the available-memory check;

- `src/fourier.c`: on periodic grids, the operators' symbols and the operators applied in Fourier space;
- `src/polar.c`, `src/cylinder.c`: the polar grid (Poisson solver, boundary conditions, cylinder diagnostics, VTK) and the flow past a cylinder (section 21);
- `tests/test_solver.c`: CPU verification and CPU–GPU parity tests;
- `tests/test_polar.c`: the polar grid's tests;
- `tests/mms.c`, `convergence.c`, `ablation.c`: the manufactured solution, the convergence study and the ablation;
- `tests/cli.sh`, `regression.sh`: the command-line and regression tests (section 16.3);
- `tools/`: post-processing scripts (spectra and cascades, snapshots, modified wavenumbers, scheme comparisons, cylinder validation).

2 Governing Equations

2.1 Nondimensional incompressible Navier–Stokes equations

For an incompressible Newtonian fluid of constant density and viscosity, the nondimensional two-dimensional Navier–Stokes equations are

$$\frac{\partial \boldsymbol{u}}{\partial t} + (\boldsymbol{u}\cdot\nabla)\boldsymbol{u} = -\nabla p + \frac{1}{\mathrm{Re}}\nabla^2\boldsymbol{u}, \tag{1}$$

$$\nabla\cdot\boldsymbol{u} = 0, \tag{2}$$

where

$$\boldsymbol{u} = (u, v)^T$$

is the velocity, p is the kinematic pressure, and

$$\mathrm{Re} = \frac{U_{\mathrm{ref}} L_{\mathrm{ref}}}{\nu}$$

is the Reynolds number.

For the default lid-driven cavity, $L_{\mathrm{ref}} = 1$ and the lid speed is $U_{\mathrm{ref}} = 1$, so the default nondimensional boundary data are

$$(u, v) = (1, 0) \qquad \text{on the moving lid,}$$
$$(u, v) = (0, 0) \qquad \text{on the other three walls.}$$

2.2 Vorticity–streamfunction formulation

Define the scalar vorticity

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}. \tag{3}$$

Taking the curl of eq. (1) eliminates pressure and yields the vorticity transport equation

$$\frac{\partial \omega}{\partial t} + u\frac{\partial \omega}{\partial x} + v\frac{\partial \omega}{\partial y} = \frac{1}{\mathrm{Re}}\left(\frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2}\right). \tag{4}$$

Introduce a streamfunction ψ by

$$u = \frac{\partial \psi}{\partial y}, \qquad v = -\frac{\partial \psi}{\partial x}. \tag{5}$$

Substitution into eq. (3) gives

$$\nabla^2 \psi = -\omega. \tag{6}$$

Thus a timestep consists conceptually of transporting ω, solving the elliptic problem eq. (6), and differentiating ψ to obtain the velocity.

2.3 Why incompressibility is naturally enforced

At the continuous level, eq. (5) gives

$$\nabla \cdot \boldsymbol{u} = \frac{\partial^2 \psi}{\partial x \partial y} - \frac{\partial^2 \psi}{\partial y \partial x} = 0.$$

The code mirrors this property discretely. Let D_x and D_y denote the two-dimensional derivative matrices. The recovered velocity is

$$\boldsymbol{u}_h = \begin{bmatrix} D_y \boldsymbol{\psi} \\ -D_x \boldsymbol{\psi} \end{bmatrix}. \tag{7}$$

The discrete divergence is therefore

$$D_x D_y \boldsymbol{\psi} - D_y D_x \boldsymbol{\psi}. \tag{8}$$

Because the operators are built as Kronecker products acting in separate coordinate directions, they commute algebraically under the implemented layout. Consequently, the divergence diagnostic is expected to be near machine precision, apart from floating-point effects and details of the boundary rows. This behavior is explicitly tested by the project.

3 Computational Grid and Data Layout

3.1 Domain and spacing

The physical domain is specified by L_x and L_y and the computational array by $n_x \times n_y$. The implementation defines

$$\Delta x = \frac{L_x}{n_x - 1}, \qquad \Delta y = \frac{L_y}{n_y - 1}, \tag{9}$$

n_x and n_y are independent; each may range from 8 through 16384.

Fields are stored as contiguous row-major arrays wrapped by the `mtrx` structure, with n_y rows of n_x values: element (i, j) lies at $y = i\,\Delta y$, $x = j\,\Delta x$ and has flat index $i\,n_x + j$, which is the ordering the Kronecker operators of eq. (17) assume. Earlier versions stored n_x rows of n_y values while the operators assumed the transpose; the two only agree when $n_x = n_y$, so non-square grids were rejected. Flattening a field is therefore a direct memory copy, and the two-dimensional derivative operations are implemented as sparse matrix–vector products.

3.2 Grid convention

The grid is nodal. Column j lies at $x_j = j\,\Delta x$, $j = 0, \ldots, n_x - 1$, and row i at $y_i = i\,\Delta y$, $i = 0, \ldots, n_y - 1$, so the first and last rows and columns of the arrays lie on the walls and n_x nodes span L_x with $n_x - 1$ intervals, as in eq. (9). All parts of the code use this convention:

1. wall velocities are imposed on the wall nodes, and the one-sided finite-difference closures are evaluated there;

2. all three Poisson solvers treat the $(n_x - 2) \times (n_y - 2)$ interior nodes as the unknowns and set $\psi = 0$ on the wall nodes (section 8.4);

3. centerline CSV output uses the node coordinates $i\,\Delta y$ and $j\,\Delta x$, and interpolates linearly onto $x = L_x/2$ (or $y = L_y/2$) when no node lies on the centerline, i.e. for an even number of nodes.

Earlier versions mixed conventions: the spacing was L/n, the FFT solver treated all $n_x \times n_y$ nodes as unknowns (placing $\psi = 0$ one cell outside the walls, while Gauss–Seidel and SOR placed it on them), and centerline coordinates were labelled $(i + \frac{1}{2})\Delta x$. Switching between solvers changed the flow, and the centerline profiles of the default case differed from Ghia et al. by up to 0.06; see section 14.12.

4 Finite-Difference Spatial Discretization

4.1 First derivative

The code supplies nominal second-, fourth-, and sixth-order centered stencils in the interior. For a uniform mesh and a sufficiently interior point,

Second order

$$f_i' \approx \frac{f_{i+1} - f_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2). \tag{10}$$

Fourth order

$$f_i' \approx \frac{f_{i-2} - 8f_{i-1} + 8f_{i+1} - f_{i+2}}{12\Delta x} + \mathcal{O}(\Delta x^4). \tag{11}$$

Sixth order

$$f_i' \approx \frac{-f_{i-3} + 9f_{i-2} - 45f_{i-1} + 45f_{i+1} - 9f_{i+2} + f_{i+3}}{60\Delta x} + \mathcal{O}(\Delta x^6). \tag{12}$$

These coefficients correspond directly to the entries created by `SDiff1()`.

4.2 Second derivative

For the Laplacian terms in the vorticity transport equation, `SDiff2()` implements:

Second order

$$f_i'' \approx \frac{f_{i-1} - 2f_i + f_{i+1}}{\Delta x^2} + \mathcal{O}(\Delta x^2). \tag{13}$$

Fourth order

$$f_i'' \approx \frac{-f_{i-2} + 16f_{i-1} - 30f_i + 16f_{i+1} - f_{i+2}}{12\Delta x^2} + \mathcal{O}(\Delta x^4). \tag{14}$$

Sixth order

$$f_i'' \approx \frac{2f_{i-3} - 27f_{i-2} + 270f_{i-1} - 490f_i + 270f_{i+1} - 27f_{i+2} + 2f_{i+3}}{180\Delta x^2} + \mathcal{O}(\Delta x^6). \tag{15}$$

4.3 Boundary closure and formal order

The requested `order` controls the widest interior stencil, but the boundary rows deliberately use narrower formulas. The closures are summarized in table 1.

Therefore, statements such as "sixth-order spatial discretization" should be read as *sixth-order centered interior stencils*, not as a demonstrated sixth-order global convergence rate. The global rate depends on the boundary closure, the Poisson discretization, the solution regularity, and the grid convention discussed above. Measured against a smooth manufactured solution with stationary no-slip walls, it is second order for every derivative order (section 14.5).

Table 1: Effective stencil order near an array boundary in the current finite-difference matrices. The pattern is mirrored at the opposite boundary.

Requested order	Location	First derivative	Second derivative
2	boundary row	1st-order forward	2nd-order one-sided
2	interior	2nd-order centered	2nd-order centered
4	boundary row	1st-order forward	2nd-order one-sided
4	first interior row	2nd-order centered	2nd-order centered
4	deeper interior	4th-order centered	4th-order centered
6	boundary row	1st-order forward	2nd-order one-sided
6	first interior row	2nd-order centered	2nd-order centered
6	second interior row	4th-order centered	4th-order centered
6	deeper interior	6th-order centered	6th-order centered

4.4 Two-dimensional operators by Kronecker product

Let d_x, d_y denote the one-dimensional first-derivative matrices, d_{xx} and d_{yy} the one-dimensional second-derivative matrices, and I_x, I_y identity matrices. The code assembles

$$D_x = I_y \otimes d_x, \qquad D_y = d_y \otimes I_x, \tag{16}$$

$$D_{xx} = I_y \otimes d_{xx}, \qquad D_{yy} = d_{yy} \otimes I_x. \tag{17}$$

This is a convenient algebraic representation of tensor-product finite differences on a Cartesian grid.

The one-dimensional matrices are assembled directly in compressed sparse row (CSR) form, and so are the two-dimensional Kronecker products. For the widest sixth-order stencil, each derivative row has only a small fixed number of nonzero entries, so sparse matrix–vector multiplication scales linearly with the number of grid points rather than quadratically or cubically in the matrix dimension.

5 Boundary Conditions and Wall Vorticity

5.1 Velocity boundary conditions

The `wall_bc` structure stores a constant (u, v) value for each of four array edges. At the beginning of every timestep, `apply_wall_bc()` writes those values directly into the edge rows and columns of the velocity arrays.

For the default cavity,

$$\boldsymbol{u}_{\text{lid}} = (1, 0), \qquad \boldsymbol{u}_{\text{other walls}} = (0, 0). \tag{18}$$

No-slip and no-penetration are therefore represented through prescribed wall velocity. The wall velocities are set in `main.c` by `u1`–`u4` and `v1`–`v4`, for the left, right, bottom and top walls in that order (`u4` is the lid).

5.2 Vorticity boundary treatment

A vorticity–streamfunction method requires a boundary value for ω that is consistent with the velocity boundary condition. The current code computes the velocity derivatives with the same finite-difference operators used elsewhere,

$$\omega|_{\partial\Omega} = (D_x v - D_y u)|_{\partial\Omega}\,, \tag{19}$$

and overwrites the boundary entries of ω before advancing the field, again at every RK4 stage, and once more at the end of the step. The update itself advances the boundary entries with the transport equation, which does not hold there, so the last overwrite replaces them with the wall vorticity of the new velocity; the field a step returns, and the one written to the VTK files, is consistent with its velocity at the walls too.

This differs from the frequently used Thom-type wall formula, which derives wall vorticity directly from a neighboring streamfunction value and wall velocity. In `cnavier`, the wall vorticity is instead obtained through one-sided/narrow derivative rows acting on the explicitly imposed velocity field. This design has the advantage of reusing one operator family for both interior and boundary derivatives, but it also means that the low-order boundary closure in table 1 directly influences the wall vorticity accuracy.

With `-wall-closure briley` (`solver_config.wall_closure` = 1), the wall vorticity is instead taken from the stream function with Briley's third-order formula. Along a wall $\psi = 0$, so $\omega = -\partial^2\psi/\partial s^2$ there, s the distance from the wall; from $\psi_0, \dots, \psi_3$ at $s = 0, h, 2h, 3h$ and the wall-tangential velocity $U = \partial\psi/\partial s(0)$,

$$\omega_0 = \frac{85\psi_0 - 108\psi_1 + 27\psi_2 - 4\psi_3}{18h^2} + \frac{11U}{3h}, \tag{20}$$

with $U = u$ on the bottom wall, $-u$ on the top, $-v$ on the left and v on the right. Thom's formula, $\omega_0 = 2(\psi_0 - \psi_1)/h^2 + 2U/h$, is the first-order member of the same family. It is used at every RK4 stage and at the end of a step like the velocity-based formula; a first step derives ψ from ω before using it. Section 14.7 measures what it changes.

5.3 Streamfunction boundary condition

The Poisson solvers impose homogeneous Dirichlet conditions on the streamfunction. For a closed cavity, setting a constant value of ψ on every wall is consistent with no normal flow; the arbitrary constant is chosen to be zero:

$$\psi|_{\partial\Omega} = 0. \tag{21}$$

Tangential wall motion is then communicated to the interior through the vorticity boundary values rather than through a nonzero streamfunction value.

6 Vorticity Transport Discretization

Define the semi-discrete right-hand side

$$\mathcal{F}(\omega; u, v) = -u \odot (D_x\omega) - v \odot (D_y\omega) + \frac{1}{\mathrm{Re}} \left(D_{xx}\omega + D_{yy}\omega\right), \tag{22}$$

where $\odot$ denotes pointwise multiplication. This is exactly the operation implemented in the CPU `dwdt()` routine and in the CUDA `rhs_kernel`.

On a periodic grid the nonlinear term can instead take the skew-symmetric form (`-advection skew`, `solver_config.advection`),

$$N_s = -\tfrac{1}{2}\left[u \odot D_x\omega + v \odot D_y\omega\right] - \tfrac{1}{2}\left[D_x(u \odot \omega) + D_y(v \odot \omega)\right], \tag{23}$$

the average of the advective and the divergence forms, which are equal in the continuum because $\nabla \cdot \mathbf{u} = 0$. The periodic centred operators are antisymmetric, $D_x^{\mathsf{T}} = -D_x$, so $\sum \omega\, D_x(u\omega) = -\sum u\,\omega\, D_x\omega$ for any u, the two halves cancel, and $\sum_{\mathrm{grid}} \omega \odot N_s = 0$: the nonlinear term conserves the discrete enstrophy $\frac{1}{2}\langle\omega^2\rangle$ exactly, as the continuum one does. Antisymmetry is all this needs. That the discrete velocity is also exactly divergence-free, $D_x u + D_y v = (D_x D_y - D_y D_x)\psi = 0$

since circulant operators commute, makes the two forms agree to the order of the scheme, as in the continuum. The advective form does not. Its error is of the order of the scheme on resolved fields, but in turbulence, where the spectrum reaches the grid cutoff, it creates enstrophy there: 23 and 45 % of the enstrophy input in the direct- and inverse-cascade runs of section 14.11. Neither form conserves the energy $\frac{1}{2}\langle u^2 + v^2 \rangle$ exactly. The energy conserved by an Arakawa-type Jacobian is $\frac{1}{2}\langle \psi\omega \rangle$, which differs from it because the Poisson operator $D_{xx} + D_{yy}$ is not $D_x^2 + D_y^2$ (section 13.4). The skew-symmetric form costs two more sparse products per evaluation and keeps the order of the stencils (section 14.8). It is the default for the turbulence cases (`forced`, `decaying`); the other cases keep the advective form, whose results they were verified with.

The convective terms are centered in the interior for all three selectable orders. Centered advection is nondissipative, which is attractive for accuracy but provides no built-in upwind damping. Consequently, timestep choice and adequate viscous resolution become important, particularly as Re increases.

7 Time Integration

7.1 Forward Euler

With `time_scheme = 1`, the update is

$$\omega^{n+1} = \omega^n + \Delta t\, \mathcal{F}(\omega^n; u^n, v^n). \tag{24}$$

After the explicit vorticity update, the Poisson equation is solved once and the velocity is reconstructed from the new streamfunction. Thus Euler requires one elliptic solve per full timestep.

Forward Euler is first-order accurate in time. Its stability region does not include a finite segment of the imaginary axis, so centered advection by itself is not Euler-stable. Viscosity may stabilize a particular discretization and timestep, but the method should be treated as the low-cost/low-order option rather than the recommended high-Reynolds-number integrator.

7.2 Classical fourth-order Runge–Kutta

With `time_scheme = 2`, the code uses classical RK4:

$$k_1 = \mathcal{F}(\omega^n), \tag{25}$$

$$k_2 = \mathcal{F}\left(\omega^n + \frac{\Delta t}{2} k_1\right), \tag{26}$$

$$k_3 = \mathcal{F}\left(\omega^n + \frac{\Delta t}{2} k_2\right), \tag{27}$$

$$k_4 = \mathcal{F}\left(\omega^n + \Delta t k_3\right), \tag{28}$$

$$\omega^{n+1} = \omega^n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \tag{29}$$

The velocity depends on the stage vorticity through eq. (6)–eq. (5). Accordingly, every call to the RK4 right-hand side performs a Poisson solve and recovers a stage-consistent (u, v) field. Four Poisson solves are therefore required for the four RK stages, followed by one final solve to make (u^{n+1}, v^{n+1}) consistent with ω^{n+1}. In total, RK4 performs five Poisson solves per timestep.

For a sufficiently smooth semi-discrete system, classical RK4 is fourth-order accurate in time. In the complete method that holds only if the boundary data move with the stages: the wall vorticity eq. (19) depends on the velocity, which changes from stage to stage, so it is recomputed from each stage's velocity before that stage's derivatives are taken (section 5.2). Earlier versions set it once per timestep, from the velocity at the start of the step. The boundary data then lagged by up to one step, RK4 converged at first order only, and at the same step its error was

about that of Euler. Measured on the lid-driven cavity (17×17, $\mathrm{Re} = 100$, ω on all nodes, walls included, at $t = 0.2$ against a run of the same scheme at $\Delta t/64$), the observed order is now 4.0 for RK4 and 1.0 for Euler, and at $\Delta t = T/64$ the error of RK4 is 1.2×10^{-8} against 2.5×10^{-2} for Euler. The test suite checks both orders. The steady state is essentially unaffected: at $t = 30$ the centerline profiles of the default case move by at most 1.2×10^{-4}, and their distance from the Ghia data changes from 0.0024/0.0074 to 0.0023/0.0073 (u/v).

8 Poisson Equation for the Streamfunction

At each velocity reconstruction, the code solves

$$\nabla^2 \psi = -\omega. \tag{30}$$

Three algorithms are available.

8.1 Five-point discrete Poisson operator

The iterative solvers and the spectral eigenvalues correspond to the conventional second-order five-point Laplacian. At an interior point, with i the row (y) and j the column (x) index as everywhere in this document,

$$\frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{\Delta x^2} + \frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{\Delta y^2} = f_{i,j}, \qquad f = -\omega. \tag{31}$$

This is an important distinction: selecting fourth- or sixth-order finite-difference operators for vorticity transport and velocity differentiation does *not* change the order of the Poisson operator. The streamfunction solve remains based on the second-order discrete Laplacian.

8.2 Gauss–Seidel

Solving eq. (31) for the center value gives the Gauss–Seidel update

$$\psi_{i,j}^{\mathrm{GS}} = \frac{\Delta y^2(\psi_{i,j+1} + \psi_{i,j-1}) + \Delta x^2(\psi_{i+1,j} + \psi_{i-1,j}) - \Delta x^2 \Delta y^2 f_{i,j}}{2(\Delta x^2 + \Delta y^2)}. \tag{32}$$

The serial CPU build sweeps lexicographically. The OpenMP build uses red–black ordering so that all nodes of one color can be updated in parallel. The CUDA implementation also uses red–black ordering.

8.3 Successive over-relaxation

SOR applies a relaxation factor β:

$$\psi_{i,j}^{(k+1)} = \beta\, \psi_{i,j}^{\mathrm{GS}} + (1 - \beta)\psi_{i,j}^{(k)}. \tag{33}$$

The default program estimates an optimal relaxation parameter from the spectral radius of Jacobi iteration on the $(n_x - 2) \times (n_y - 2)$ interior nodes. With the weights of the five-point stencil (Δy^2 on the x neighbours, Δx^2 on the y neighbours), the slowest Jacobi mode is the lowest sine mode in each direction, so

$$\rho = \frac{\Delta y^2 \cos\left(\dfrac{\pi}{n_x - 1}\right) + \Delta x^2 \cos\left(\dfrac{\pi}{n_y - 1}\right)}{\Delta x^2 + \Delta y^2}, \tag{34}$$

$$\beta = \frac{2}{1 + \sqrt{1 - \rho^2}}. \tag{35}$$

which reduces to $\rho = \cos(\pi/(n-1))$ on a square grid with $\Delta x = \Delta y$. The five-point stencil is consistently ordered in both the lexicographic and the red–black sweep, so Young's optimum β applies to the CPU and the GPU solvers alike.

The default maximum iteration count is 10000 and the shipped convergence threshold is 10^{-3}.

Stopping metric. The implementation function named `error()` computes

$$E^{(k)} = \sum_{i,j} \left| \psi_{i,j}^{(k+1)} - \psi_{i,j}^{(k)} \right|, \tag{36}$$

not a residual norm and not a root-sum-square norm. The console string currently labels this quantity "RSS error", but scientifically it is an L_1 iterate-change metric. Verification tests separately compute the actual five-point Poisson residual.

Because the default tolerance can stop the iteration before full convergence, the sweep ordering can affect the returned approximation. This is why the serial lexicographic CPU result and the red–black GPU result need not be identical at the shipped tolerance; they approach the same solution as the tolerance is tightened.

8.4 Direct FFT/DST-I solver

The default `poisson_type = 3` uses a two-dimensional discrete sine transform of type I (DST-I) of the $m_x \times m_y$ interior nodes, $m_x = n_x - 2$, $m_y = n_y - 2$. A DST-I of length m implies zeros at indices -1 and m, which are exactly the wall nodes, so the solver imposes $\psi = 0$ on the wall nodes like the iterative solvers and solves the same discrete problem. For each spectral mode (p, q), the standard second-difference Laplacian has eigenvalue

$$\lambda_{p,q} = \frac{2\cos\left(\frac{\pi p}{m_x+1}\right) - 2}{\Delta x^2} + \frac{2\cos\left(\frac{\pi q}{m_y+1}\right) - 2}{\Delta y^2}, \tag{37}$$

with $p = 1, \ldots, m_x$ and $q = 1, \ldots, m_y$; note $m_x + 1 = n_x - 1$.

The algorithm is:

1. transform the right-hand side with a two-dimensional DST-I;
2. divide each transformed coefficient by $\lambda_{p,q}$;
3. apply the DST-I again;
4. multiply by the inverse transform normalization

$$\frac{1}{4(m_x+1)(m_y+1)} = \frac{1}{4(n_x-1)(n_y-1)}.$$

On the CPU the two-dimensional DST-I is applied as two passes of one-dimensional FFTW3 `RODFT00` transforms, first along the rows and then along the columns, in batches of eight rows or columns. The plans are created once at startup with `FFTW_ESTIMATE` and reused. Two choices make the result deterministic. `FFTW_ESTIMATE` picks plans without timing them; `FFTW_MEASURE`, used previously, could pick a different plan on every run, and different plans round differently, so repeated runs at 513×513 differed in the last bits. And every row is transformed by the same plan whatever the number of threads, so serial and OpenMP builds give bit-identical results. The asymptotic cost is $\mathcal{O}(N \log N)$ with $N = n_x n_y$, compared with a potentially large number of $\mathcal{O}(N)$ sweeps for Gauss–Seidel or SOR.

Once Gauss–Seidel or SOR has converged, all three solvers return the same ψ; the test suite checks agreement to 10^{-9} on a random right-hand side.

The FFT solver is "direct" with respect to the *discrete second-order Poisson problem*: once the transform is performed, no iterative convergence tolerance is required. It is not an exact solver for the continuous Poisson equation; the continuous truncation error remains that of the discrete Laplacian and the grid representation.

8.5 Forcing and drag

On the periodic grid five built-in terms drive and damp the flow,

$$\frac{\partial \omega}{\partial t} = -\mathbf{u}\cdot\nabla\omega + \nu\nabla^2\omega - \alpha\omega + f_K - \nu_h(-\nabla^2)^p\omega - \alpha_h\psi, \qquad f_K = -Ak\cos(ky), \quad k = 2\pi n/L_y, \tag{38}$$

plus a random kick before every step (`solver_config.forcing`, `src/forcing.c`). The linear drag $-\alpha\omega$ removes the energy the inverse cascade carries to the largest scales. The hyperviscosity $-\nu_h(-\nabla^2)^p\omega$ applies the discrete Laplacian $D_{xx}+D_{yy}$ p times (on the GPU in Fourier space, section 12.3); its damping rate $\nu_h Q^p$ is negligible except near the grid cutoff, so it removes the enstrophy of the direct cascade there and leaves the range above almost inviscid, which plain viscosity at an affordable resolution cannot do. The hypodrag $-\alpha_h\psi = -\alpha_h(-\nabla^2)^{-1}\omega$ uses the stream function of the stage; its damping rate α_h/Q confines it to the largest scales, where it stops the inverse cascade instead of damping it along the way as linear drag does. The time-step limit of section 10 includes all the damping terms. A periodic mode with eigenvalue Q of $-(D_{xx}+D_{yy})$ decays at $\sigma(Q) = Q/\mathrm{Re} + \nu_h Q^p + \alpha + \alpha_h/Q$, which is convex for $Q>0$, so its maximum over the modes is at the largest Q, the $\lambda_{\max}$ of section 10, or with hypodrag possibly at the smallest non-zero one, the symbol of D_{xx} or D_{yy} at the lowest wavenumber. The run is refused unless $\Delta t \le \rho/\max\sigma$. For each term the limit is sharp: the single mode that sets it (on which the nonlinear term vanishes) decays at 0.95 times the limit and grows at 1.05 times, with Euler and RK4. f_K is the curl of the Kolmogorov body force $A\sin(ky)\,\hat{\mathbf{x}}$; its laminar state is $\omega = f_K/(\nu k^2+\alpha)$, on which the nonlinear term vanishes. The random kick adds

$$\delta\omega = \sum_m a_m \cos(\mathbf{k}_m\cdot\mathbf{x} + \phi_m) \tag{39}$$

over the wavevectors $\mathbf{k}_m$ of the half plane with $|\,|\mathbf{k}_m|/\Delta k - k_f| \le \delta_k$, with phases ϕ_m drawn anew each step. With A_m and Q_m the symbols of section 13.4, a mode carries the discrete energy $A_m a_m^2/(4Q_m^2)$, and distinct modes are orthogonal on the grid, so $a_m = 2Q_m\sqrt{\varepsilon\,\Delta t/(MA_m)}$ makes each kick carry exactly $\varepsilon\,\Delta t$ of the solver's own kinetic energy. Since the phases are independent of the flow, the cross term with ω averages to zero and the energy is injected at rate ε: white-in-time forcing. The phases come from a seeded generator on the host, so the CPU and GPU solvers make the same kicks; only the phases go to the GPU each step. The integrals of section 13.4 gain the energy input $I = \langle u\,A\sin(ky)\rangle + \varepsilon$. For the random forcing, ε is the *expected* input: a kick also changes E by $\langle\mathbf{u}\cdot\delta\mathbf{u}\rangle$, which averages to zero but not on a single step, and divided by Δt it grows noisier as $\Delta t\to 0$, as white noise does. The energy budget therefore holds on average, and at a statistically steady state $I_{\mathrm{disc}} = \sum_b D_E + 2\alpha E$, which is $I = 2\nu Z + 2\alpha E$ when the flow is resolved. The Kolmogorov work is the physical, continuum expression. The rate at which the implemented source f_K changes the discrete energy differs from it: f_K is a single mode, its stream function is $\psi_f = -(Ak/Q)\cos(ky)$ and D_y gives it the velocity $(Ak\tilde{k}_1/Q)\sin(ky)$, with $\tilde{k}_1$ and Q the symbols of D_y and $-D_{yy}$ at k, so the discrete injection is $(k\tilde{k}_1/Q)\,\langle uA\sin(ky)\rangle = 1+\mathcal{O}(h^p)$ times the work. `integrals.csv` has both, I and I_{disc}; with I_{disc} the discrete budget $\mathrm{d}E/\mathrm{d}t = \sum T_E - \sum(D_E+F_E) + I_{\mathrm{disc}}$ is exact. With random forcing the time integration is RK4 for the deterministic terms plus an additive kick per step, so the trajectories are not fourth order in time.

8.6 Fourth-order compact Poisson operator

With `-poisson-order 4` (`solver_config.poisson_order`), the FFT solver discretises $\nabla^2\psi = f$ with the compact nine-point ("Mehrstellen") operator instead of the five-point one,

$$\left[\delta_x^2 + \delta_y^2 + \frac{\Delta x^2 + \Delta y^2}{12}\,\delta_x^2\delta_y^2\right]\psi = \left[1 + \frac{\Delta x^2\delta_x^2 + \Delta y^2\delta_y^2}{12}\right] f, \tag{40}$$

with δ_x^2, δ_y^2 the second differences of eq. (31). It is fourth order for any Δx, Δy. The same sine transform diagonalises it: mode (p, q) has eigenvalue $\mu_p + \mu_q + \frac{\Delta x^2+\Delta y^2}{12}\mu_p\mu_q$, where μ_p and μ_q are the two terms of eq. (37), the eigenvalues of δ_x^2 and δ_y^2, so the cost is that of the five-point solve. The right-hand side is $f + (f_E + f_W + f_N + f_S - 4f)/12$, whatever the spacing. Next to a wall it needs $f = -\omega$ on the wall nodes; these are taken from the quadratic through the first three interior nodes along the wall normal, $3f_1 - 3f_2 + f_3 = f_0 + \mathcal{O}(h^3)$, rather than from the wall vorticity, so the Poisson solve still reads only the interior of ω and does not couple back to the wall closure. The extrapolation error, $\mathcal{O}(h^3)$ in the first interior row only, does not lower the order. On a test with a non-zero Laplacian on the walls the operator converges at about five (4.9 from 33^2 to 129^2). The option needs the FFT solver; Gauss–Seidel and SOR keep the five-point operator.

8.7 Doubly periodic boundaries

With `solver_config.periodic` set, the domain $[0, L_x) \times [0, L_y)$ is periodic in both directions and has no walls. The grid has n_x points spaced $\Delta x = L_x/n_x$ (the point at $x = L_x$ is the point at $x = 0$), and likewise in y. Three things change.

- **Derivative operators.** Every row of `SDiff1_periodic` and `SDiff2_periodic` is the centered interior stencil of the chosen order, with the indices taken modulo n, so there are no one-sided rows.
- **Poisson operator.** The Laplacian is $D_{xx} + D_{yy}$, built from the same second-derivative operators as the transport, so the Poisson solve has the order of the derivatives rather than second order. Both operators are circulant, so the two-dimensional discrete Fourier transform diagonalises them: mode (k_x, k_y) has eigenvalue $\lambda_x(k_x) + \lambda_y(k_y)$, where $\lambda_x(k) = \sum_m c_m \cos(2\pi k m/n_x)$ is the symbol of the stencil with coefficients c_m, read from the operator's first row. The solver transforms $-\omega$ with a real-to-complex FFT (FFTW, or cuFFT on the GPU), divides by the eigenvalues, and transforms back. The mean mode has eigenvalue zero: the mean of ψ is set to zero and the mean of ω is ignored, as a periodic domain carries no net circulation.
- **No wall terms.** The wall-velocity and wall-vorticity updates are skipped. The periodic grids need the FFT solver; Gauss–Seidel and SOR are not offered.

The command line provides periodic cases on the unit square, among them (with those of section 8.5) decaying turbulence from a random-phase field whose modal amplitudes have the shell envelope $E(k) \propto (k/k_0)^4 e^{-2(k/k_0)^2}$ (the shell sums of the lattice modes fluctuate about it with the number of modes per shell), normalised to $E = \frac{1}{2}$ (`-case decaying`, `-peak-k`); the Taylor–Green vortex $\psi = \sin(kx)\sin(ky)\,e^{-2k^2t/\mathrm{Re}}/k$, $k = 2\pi$, an exact solution whose error the program prints at the end, and the double shear layer of Bell, Colella and Glaz (1989), which rolls up into vortices at high Reynolds numbers.

8.8 Compact schemes on the periodic grid

With `-order compact6` (`FD_COMPACT6`), the periodic cases use Lele's sixth-order tridiagonal compact schemes [5] for the first and the second derivatives,

$$\tfrac{1}{3}f'_{i-1} + f'_i + \tfrac{1}{3}f'_{i+1} = \tfrac{14}{9}\,\frac{f_{i+1}-f_{i-1}}{2h} + \tfrac{1}{9}\,\frac{f_{i+2}-f_{i-2}}{4h}, \tag{41}$$

$$\tfrac{2}{11}f''_{i-1} + f''_i + \tfrac{2}{11}f''_{i+1} = \tfrac{12}{11}\,\frac{f_{i+1}-2f_i+f_{i-1}}{h^2} + \tfrac{3}{11}\,\frac{f_{i+2}-2f_i+f_{i-2}}{4h^2}, \tag{42}$$

that is $Af' = Bf$, with symbols ($\theta = kh$)

$$k^*h = \frac{\frac{14}{9}\sin\theta + \frac{1}{18}\sin 2\theta}{1 + \frac{2}{3}\cos\theta}, \qquad (k^*h)^2 = -\frac{\frac{24}{11}(\cos\theta - 1) + \frac{3}{22}(\cos 2\theta - 1)}{1 + \frac{4}{11}\cos\theta}. \tag{43}$$

On a periodic grid $A^{-1}B$ is a circulant matrix, and the code builds it as one, so every part of the solver uses it unchanged as a sparse operator: the transport, the velocity recovery, the Poisson operator, the diagnostics, the stability limit and the GPU. The circulant inverse of $A = \mathrm{tridiag}(\alpha, 1, \alpha)$ is

$$g_j = \frac{r^j + r^{n-j}}{(1-r^n)\sqrt{1-4\alpha^2}}, \qquad 0 \le j < n, \qquad r = \frac{\sqrt{1-4\alpha^2}-1}{2\alpha}, \tag{44}$$

the periodic sum of the infinite line's $r^{|j|}/\sqrt{1-4\alpha^2}$. Convolving it with the five-point stencil of B gives the entries of $A^{-1}B$ exactly, even the smallest. They decay like $|r|^j$: 0.38 per point for the first derivative and 0.19 for the second. Entries below 10^{-16} of the largest are dropped, which leaves 78 and 45 entries per row for $n \ge 128$ and changes the symbol at round-off level. (A direct inverse DFT of the symbol accumulates round-off of order $10^{-16}\sqrt{n}$ in every entry, and then no entry can be dropped.) The matrices match the symbols on every Fourier mode of the grid to 4×10^{-14}, annihilate constants, and the first derivative is antisymmetric, so the skew-symmetric form of eq. (23) conserves enstrophy with them too.

What the compact schemes buy is resolution: the fraction of the wavenumbers up to Nyquist that each scheme differentiates to 1 % and 10 % (`tools/modified_wavenumber.py`, fig. 1):

	first derivative		second derivative	
	1 %	10 %	1 %	10 %
order 2	0.08	0.25	0.11	0.36
order 4	0.24	0.44	0.32	0.60
order 6	0.35	0.54	0.45	0.71
compact 6	0.50	0.70	0.56	0.81

Building the operator as a banded matrix makes it exact and available everywhere at once, but not cheap. The rows are 78 and 45 wide instead of 7, and in y they span as many grid rows. A step on the GPU costs about 21 times the explicit one: 269 against 12.3 ms at 512^2, and 66 against 3.2 at 256^2. Applied in Fourier space (section 8.9), the same operators cost what every other scheme costs there.

8.9 Operators in Fourier space and pseudospectral differentiation

On a periodic grid every derivative operator is circulant, so it is diagonal in Fourier space, with its symbol as the eigenvalues. The symbols are read from the first row of the 1-D operators, as $\sum_j c_j e^{i\theta j}$ (`periodic_symbols_of()`). The same table now gives everything that needs them: the Poisson eigenvalues, A/Q for the spectra, the kick amplitudes and the Kolmogorov factor. With `-operators fourier` (`solver_config.fourier`, `src/fourier.c` and its counterpart in `cudasolver.cu`), the solver applies the operators that way instead of as sparse products:

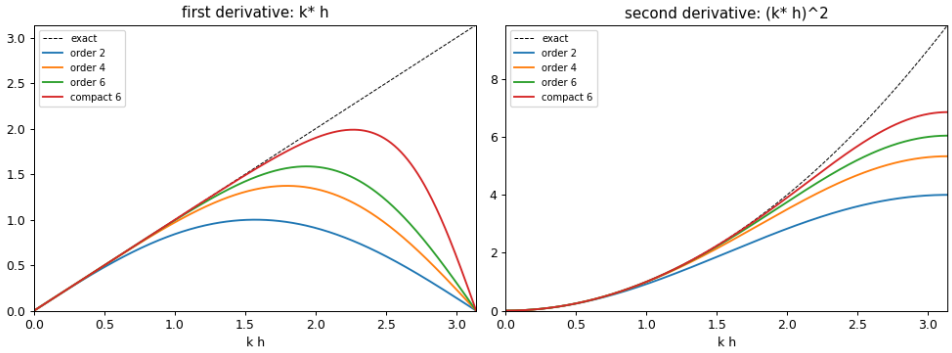


Figure 1: Modified wavenumbers of the periodic schemes.

- the velocity: one forward transform of ω, then $\hat{\psi} = -\hat{\omega}/(\tilde{D}_{xx} + \tilde{D}_{yy})$, and inverse transforms of $\hat{\psi}$, $\tilde{D}_y\hat{\psi}$ and $-\tilde{D}_x\hat{\psi}$;
- the transport: one forward transform, and inverse ones of $\tilde{D}_x\hat{\omega}$, $\tilde{D}_y\hat{\omega}$ and $(\tilde{D}_{xx} + \tilde{D}_{yy})\hat{\omega}$;
- the skew-symmetric correction (two forward transforms and one inverse), the hyperviscosity, the palinstrophy, the continuity check and the spectra.

The 2-D sparse operators are then not built. The operators are the same matrices, so the results equal the sparse ones to round-off. Twenty steps with every term on agree to 10^{-14} for explicit order 6 and for the compact scheme, and so do the integrals and the spectra; the GPU matches the CPU to 6×10^{-15}. A decaying run with the compact scheme reproduces the banded run's spectrum to 4×10^{-12}.

The table also allows operators that have no sparse form. `-order spectral` (`FD_SPECTRAL`) uses the symbols ik and $-k^2$ of pseudospectral differentiation (with the first-derivative symbol zero at the Nyquist wavenumber, so that real fields stay real), and needs the Fourier operators. Its 1-D matrices are dense and serve only for the symbols and the stability limit. With exact symbols $A = Q$, so the energy of a mode is $\frac{1}{2}|\hat{\psi}|^2 Q$ and the two discrete energies coincide. `-dealias` adds Orszag's 2/3 rule: the operators keep only $|k| < n/3$ along each axis, and ω is filtered at the end of each step. Products of the kept fields then alias only onto cut modes, so the nonlinear term is a Galerkin truncation and conserves energy and enstrophy exactly in either form; the net transfers are 10^{-14} of the largest shell transfer. `-pad` dealiases by padding instead (the 3/2 rule [10]): the spectra of u, v, $\partial_x\omega$ and $\partial_y\omega$ are zero-padded to a grid 3/2 as fine (the Nyquist modes left out), the product is formed there, and its spectrum is truncated back (`fourier_nonlinear_padded()`). Products of two modes with $|k| < n/2$ then alias only onto wavenumbers of at least $n/2$, so every mode of the grid but the Nyquist ones carries the Fourier–Galerkin nonlinear term. `make test` checks it against the direct convolution of the modal coefficients on 10×8 and 9×7 grids (7×10^{-15}), and the net transfers in either form (10^{-14}). The cost is four inverse and one forward transform on $(3n/2)^2$ points per evaluation. On the manufactured solution the pseudospectral error at 32^2 is 3.5×10^{-9}, against 1.1×10^{-4} with the compact scheme, and it falls by 16 when Δt is halved: it is the time integration's.

In Fourier space every scheme costs the same, so the scheme is chosen by what it resolves and by its advective limit. Per GPU step:

ms per step	explicit 6, sparse	explicit 6	compact 6	spectral	spectral, 2/3
256^2	3.6	4.8	4.8	4.8	5.0
512^2	12.8	17.7	17.7	17.7	18.4
1024^2	51.8	78.5	78.6	78.5	82.0

(decaying turbulence, skew-symmetric form). The transforms cost more than the explicit seven-point stencils, so explicit order 6 stays sparse by default. The compact scheme is 15 times faster than in its banded form.

9 Complete Timestep Algorithms

9.1 Euler sequence

One CPU Euler timestep performs the following operations:

1. impose wall velocities on u and v;
2. compute $D_y u$ and $D_x v$;
3. overwrite boundary vorticity using $\omega = D_x v - D_y u$;
4. compute $D_x\omega$, $D_y\omega$, $D_{xx}\omega$, and $D_{yy}\omega$;
5. update ω by forward Euler;
6. solve $\nabla^2\psi = -\omega$;
7. recover $u = D_y\psi$ and $v = -D_x\psi$;
8. impose the wall velocities on copies of u and v and overwrite the boundary vorticity with $D_x v - D_y u$ of those copies, since the update advanced the boundary entries with the transport equation, which does not hold there. u and v themselves keep the values from ψ, whose divergence the continuity check measures.

9.2 RK4 sequence

One RK4 timestep begins with the same wall-velocity and wall-vorticity update. Each of the four stage right-hand-side evaluations then:

1. solves the Poisson equation for the stage streamfunction from the interior stage vorticity;
2. recovers a stage-consistent velocity field and imposes the wall velocities on it;
3. sets the wall vorticity of the stage from that velocity, eq. (19);
4. differentiates the stage vorticity;
5. evaluates the nonlinear convective and viscous terms.

After combining k_1 through k_4, a fifth Poisson solve recovers velocity for the completed step, and the step ends like the Euler step: the wall velocities are imposed on copies of u and v, and the boundary vorticity is overwritten with $D_x v - D_y u$ of those copies.

The first stage of the next step would solve the Poisson equation for the same interior vorticity as the fifth solve of this one. With the FFT solvers, which read only the interior of ω (all of it on a periodic grid), that solve is skipped and the velocity is recovered from the stored ψ. RK4 thus does four solves per step after the first, and the results are bitwise the same. The CPU compares the interior of ω with a copy of the field last solved for, made only after the solve that ends a step (not after the stage solves, nor in runs with random kicks, which change ω every step). The GPU, which owns its fields, keeps a flag that every change to the interior of ω clears. The random kick changes ω at the start of every step, so forced runs still do five solves. The iterative solvers start from the previous ψ, so a second solve can converge further, and they are never skipped. The gain shows what a solve costs. The cavity's sine-transform solve is a large

share of a step: 11–12 % faster on the CPU (129^2, 257^2), 8–12 % on the GPU (257^2, 513^2). The periodic solve is cheap: 2–3 % on the GPU (512^2, 1024^2) and on the CPU (256^2) at most a few per cent, within the run-to-run noise.

9.3 Pseudocode

```
construct sparse DX, DY, DX2, DY2
allocate reusable solver workspace
initialize u, v, omega

for each timestep:
    impose wall velocities on u, v
    omega_wall = DX*v - DY*u on array edges

    if Euler:
        rhs = -u*(DX*omega) - v*(DY*omega)
              + (1/Re)*(DX2*omega + DY2*omega)
        omega += dt*rhs
        solve Laplacian(psi) = -omega
        u = DY*psi
        v = -DX*psi

    if RK4:
        for each RK stage:
            solve Laplacian(psi_stage) = -omega_stage   (interior)
            u_stage = DY*psi_stage
            v_stage = -DX*psi_stage
            impose wall velocities on u_stage, v_stage
            omega_stage_wall = DX*v_stage - DY*u_stage on array edges
            k_stage = RHS(omega_stage, u_stage, v_stage)
        omega = RK4 combination
        solve Laplacian(psi) = -omega
        u = DY*psi
        v = -DX*psi

    (u_w, v_w) = (u, v) with wall velocities imposed
    omega_wall = DX*v_w - DY*u_w on array edges

    evaluate discrete divergence DX*u + DY*v
    optionally write VTK
```

10 Stability and Timestep Selection

Because both time integrators are explicit, the timestep is restricted by the spectra of the discrete advection and diffusion operators.

10.1 Implemented Courant check

Before starting a run, `main.c` computes, for the default moving-wall velocity,

$$C_x = U_{\text{lid}}\frac{\Delta t}{\Delta x}, \qquad C_y = U_{\text{lid}}\frac{\Delta t}{\Delta y}, \tag{45}$$

and aborts if either exceeds `max_co = 1`, where U_{lid} is the largest wall speed. This is a useful sanity check but should not be interpreted as a complete stability proof. The actual interior velocity can differ from the lid velocity, and diffusion imposes a separate explicit restriction, which is checked as described next.

10.2 Implemented viscous limit

`max_stable_dt()` in `fluiddyn.c` bounds the timestep for the viscous term $\mathrm{Re}^{-1}(D_{xx} + D_{yy})$. The interior second-derivative stencils are centered with coefficients of alternating sign, so the sum of their magnitudes is the value of the stencil symbol at the highest grid frequency, i.e. the largest eigenvalue magnitude:

$$\lambda_{\max} = \frac{1}{\mathrm{Re}}\Big(\sum_k |a_k^x| + \sum_k |a_k^y|\Big), \tag{46}$$

taken from the middle row of the one-dimensional operators (which is also the middle row of the two-dimensional ones), so the check runs before the two-dimensional operators are assembled. The run aborts unless

$$\Delta t \le \frac{\rho}{\lambda_{\max}}, \qquad \rho = \begin{cases} 2 & \text{forward Euler,} \\ 2.785293\ldots & \text{classical RK4,} \end{cases} \tag{47}$$

ρ being the extent of each method's stability region along the negative real axis, and unless the Courant condition holds; the message names both limits and suggests the smaller one, rounded down so that the suggestion itself passes. For forward Euler the classical centered advection–diffusion condition $\Delta t \le 2\nu/U^2 = 2/(\mathrm{Re}\,U^2)$ is checked as well, but only as a warning, since it is conservative here (below); when an Euler run is refused, the suggested Δt is the smallest of all three limits, so that following it does not trigger the warning. Measured on the lid-driven cavity, the viscous bound is sharp: for orders 2, 4 and 6, both schemes and $n = 64$ and 128, runs at 0.95 times the bound stay bounded for 3000 steps and runs at 1.05 times diverge. The advection condition is conservative: at $\mathrm{Re} = 1000$, $n = 64$, Euler stayed bounded up to about three times it. At the defaults ($n = 64$, $\mathrm{Re} = 100$, sixth order) the bounds are $\Delta t \le 0.00580$ for RK4 and $\Delta t \le 0.00416$ for Euler.

10.3 Diffusive scaling

For the simple second-order Laplacian with forward Euler, the familiar two-dimensional diffusion restriction is approximately

$$\frac{1}{\mathrm{Re}}\Delta t\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right) \lesssim \frac{1}{2}. \tag{48}$$

For $\Delta x = \Delta y = h$, this reduces to

$$\Delta t \lesssim \frac{\mathrm{Re}\,h^2}{4}. \tag{49}$$

The precise limit changes with the higher-order second-derivative stencil and with RK4, but the important scaling remains

$$\Delta t = \mathcal{O}(h^2) \tag{50}$$

when explicit viscous stability controls the calculation. The repository benchmarks therefore use a timestep proportional to $1/n^2$.

10.4 Centered advection

The interior convective derivative is centered. Such a discretization contributes nearly imaginary eigenvalues for pure linear advection. RK4 has a finite stability interval along the imaginary axis and is therefore much better suited than Euler to centered advection. At high Reynolds number, however, neither time integrator substitutes for spatial stabilization if the grid is too coarse to resolve the vorticity gradients.

10.5 Recommended practice

For scientific runs:

- prefer RK4 over Euler unless Euler is being used deliberately for a comparison;
- reduce Δt under grid refinement, normally at least proportionally to h^2 until temporal error is demonstrated to be negligible;
- verify timestep independence by repeating selected calculations with $\Delta t/2$;
- monitor divergence, extrema, and visual smoothness of the vorticity field;
- treat a run that merely passes the implemented $C \leq 1$ check as *admissible input*, not automatically as a stable or converged simulation.

11 Sparse Linear Algebra

The dominant derivative operation is a CSR sparse matrix–vector multiplication

$$\boldsymbol{y} = A\boldsymbol{x}. \tag{51}$$

For each row, `spmv()` loops only over the nonzero entries stored between consecutive elements of `row_ptr`. Since the finite-difference bandwidth is fixed, the cost is $\mathcal{O}(N)$ for $N = n_x n_y$.

The main dense field arrays are contiguous. Reusable flat buffers and RK4 work arrays are allocated once before the time loop; no per-stage dynamic memory allocation is needed. This improves both CPU cache behavior and GPU transfer strategy.

12 Parallel Implementations

12.1 Backend structure

A run is described by one `solver_config` (`fluiddyn.h`): grid size and spacing, Reynolds number, timestep, time scheme, Poisson solver settings, wall velocities and the four derivative operators. Both backends are initialised from it, and each keeps its own copy, taken at that moment: the CPU solver in its workspace (`rk4_ctx`, which also holds the work arrays and FFT plans) and the GPU solver (`gpu_init`), which uploads the operators and fields to the device. Changing the caller's `solver_config` afterwards affects neither, and a test checks that both ignore such a change alike. `backend.c` chooses between them once, at startup, and exposes the same operations for both (one timestep, the divergence diagnostic, the host fields), so the time loop in `main.c` does not depend on where it runs. On the CPU the host fields are the solver's own arrays; on the GPU they are copies filled from the device when asked for. A GPU run allocates no CPU solver workspace, and the memory check counts only what the chosen backend allocates.

12.2 OpenMP

Building with

```
make OPENMP=1
```

enables shared-memory parallel loops in the principal CSR operations, explicit right-hand-side loops, and suitable Poisson operations, including the batches of one-dimensional sine transforms. Loops over fewer than 2048 grid points (`OMP_MIN_WORK`) stay serial, since starting the threads would cost more than they save. Unless `OMP_NUM_THREADS` or a thread placement (`OMP_PROC_BIND`, `OMP_PLACES`) is set, the program uses one thread per physical core in its affinity mask instead of

the OpenMP default of one per logical CPU: with two hardware threads per core the second adds nothing to these bandwidth-limited loops, and on an idle machine 16 threads on 8 cores were 25–40% slower than 8. Surplus threads are far more costly when other programs occupy the cores, since every parallel loop ends at a barrier where all threads wait for the slowest. Because the CSR products and transforms are limited by memory bandwidth, the speed-up levels off at about two from four threads on. Iterative elliptic solves use red–black coloring because a naive parallelization of lexicographic Gauss–Seidel would violate its data dependence. Results are bitwise identical to the serial build with any number of threads, except with Gauss–Seidel/SOR, where they are identical among OpenMP runs (the parallel sweep order differs from the serial one).

On a busy machine fewer threads are faster. A step has about 70 parallel loops, and a thread that is descheduled holds all the others up at the next barrier: with other jobs running, 16 threads took 577 ms per step at 1025^2, and the 8-thread default slowed down severalfold too. On a shared machine set `OMP_NUM_THREADS` below the number of free cores, or `OMP_WAIT_POLICY=passive`, so that waiting threads sleep instead of spinning. With a thread placement set, the OpenMP default is kept, one thread per logical CPU; combine it with `OMP_NUM_THREADS` to choose the count, e.g. `OMP_PLACES=cores OMP_NUM_THREADS=8`. OpenMP comes with GCC (`libgomp`); Apple's Clang needs either GCC from Homebrew (`CC=gcc-14 make OPENMP=1`) or `libomp`.

Changing the sweep order is numerically relevant when the solver is stopped before convergence. Therefore, at a loose iterate-change tolerance, serial Gauss–Seidel/SOR and OpenMP red–black Gauss–Seidel/SOR can return slightly different intermediate approximations even though they converge to the same discrete solution when the tolerance is sufficiently tight.

12.3 CUDA

Building with

```
make CUDA=1
```

adds a GPU backend. It needs an NVIDIA GPU with a working driver and the CUDA toolkit (`nvcc` and cuFFT; on Ubuntu/Debian the `nvidia-cuda-toolkit` package is enough). `nvcc` is taken from `PATH`, falling back to `/usr/local/cuda/bin/nvcc`; `NVCC` or `CUDA_HOME` point elsewhere (`make CUDA=1 CUDA_HOME=/opt/cuda`). A CUDA build uses the GPU by default and `-cpu` runs the CPU path from the same binary; started on a machine without a usable GPU, it says so and runs on the CPU. The four CSR derivative matrices and all primary fields are uploaded once and remain on the device throughout the time loop. Data returns to the host only for the VTK files, the continuity extrema and the final centerline profiles. CUDA kernels implement:

- wall velocity and wall-vorticity updates;
- CSR sparse derivatives;
- Euler and RK4 stage algebra;
- velocity recovery;
- red–black Gauss–Seidel/SOR;
- the FFT-based Poisson solver;
- the divergence diagnostic and reductions;
- on periodic grids, the forcing and damping terms and the skew-symmetric nonlinear term, and the spectra and fluxes of section 13.4.

Two of these use the periodic transforms the Poisson solver already plans. The hyperviscosity $(-L)^p\omega$ is $Q^p\hat{\omega}$ in Fourier space, with Q the eigenvalues of $-(D_{xx}+D_{yy})$ the solver holds. One real-to-complex and one complex-to-real transform replace the $2p$ sparse products per stage the CPU does. The operator is the same circulant matrix, so the two agree to round-off: 3×10^{-15} after 20 steps in which the hyperviscosity damps $\langle\omega^2\rangle$ to 9 % ($p=2$) and 32 % ($p=4$). The spectra transform u, v, ω and N with cuFFT and sum each shell in one thread block, over a list of the shell's modes, with a reduction in a fixed order, so the result is deterministic. Only the $8\times$(number of shells) sums are copied to the host, not the fields; they match the host computation to 3×10^{-15} for both nonlinear forms with every damping term. On 512^2 forced turbulence (hyperviscosity of order 4, spectra every 50 steps) a step takes 16.2 ms instead of 23.1, and a spectrum frame about 5 ms instead of 20.

All solver arithmetic is double precision.

12.4 GPU sine transform through odd extension

cuFFT does not expose a native DST-I. The CUDA backend uses the identity that the DST-I of $x_1,\dots,x_m$ is $-\operatorname{Im}$ of the real FFT of the odd extension $(0,x_1,\dots,x_m,0,-x_m,\dots,-x_1)$ of length $2(m+1)$. The two-dimensional transform is done in two passes of batched one-dimensional transforms, as on the CPU: the odd extensions of the n_y-2 interior rows (length $2(n_x-1)$) are transformed together, the sine modes are read off and extended along the columns (length $2(n_y-1)$), and those are transformed together. The code then divides by the same eigenvalues as eq. (37), applies the transform again, and scales by the DST normalization. An earlier version transformed one two-dimensional odd extension of size $2(n_y-1)\times2(n_x-1)$, four times the grid; the batched passes are about 1.3 times faster from 129×129 upwards.

The red–black Gauss–Seidel and SOR solvers evaluate their stopping rule on the device: each sweep's sum of $|\Delta\psi|$ is reduced and compared with the tolerance by a kernel, sweeps after the one that meets it do nothing, and the host reads the outcome once every 32 sweeps. The iteration count and the result are the same as with a check after every sweep.

This makes the GPU FFT algorithm mathematically equivalent to the CPU FFTW DST-I solver, up to floating-point round-off and the details of the FFT implementations. On the default case (Re = 100, 64×64, RK4 + FFT, 6000 steps) the GPU and CPU runs write identical centerline profiles and byte-identical VTK files. With Gauss–Seidel/SOR the answer depends on the CPU build compared with: the default tolerance stops the iteration well before convergence, so the sweep order shows. Against the `OPENMP=1` build, which also sweeps red–black, the GPU takes the same number of iterations and agrees to round-off; against the serial build, which sweeps lexicographically, it does not (with SOR on the default case, 67 against 90 iterations on the first solve and centerline velocities about 2×10^{-5} apart). The two meet only when the tolerance is tight enough for the iteration to converge.

13 Diagnostics and Output

13.1 Discrete continuity diagnostic

After every timestep, the CPU path computes

$$\epsilon_{\nabla\cdot u} = D_x u + D_y v \tag{52}$$

and prints its maximum and minimum. The CUDA path computes the same field on-device and reduces it to its extrema. Since velocity is recovered from a streamfunction, these values are expected to remain close to floating-point round-off for the compatible operator pair.

This diagnostic should be interpreted as a consistency check on the streamfunction-to-velocity machinery. It is not, by itself, evidence that the momentum/vorticity equations are spatially converged.

13.2 VTK output

By default, vorticity is written to legacy ASCII VTK files in `output/` (`STRUCTURED_POINTS` with the grid spacing; on the polar grid `STRUCTURED_GRID` with the Cartesian coordinates of the points, section 21). Each field is its own numbered series (`vorticity-1-0.vtk`, `vorticity-1-1.vtk`, ...). The first time a run writes a series, it deletes the files of that series an earlier run left in `output/`, so a series always comes from one run; other files are left alone. Other fields can be written by adding `printvtk()` calls in `main.c`. The output interval is configurable and may be disabled with

```
./cnavier --output-interval 0
```

for performance measurements.

13.3 Centerline profiles

At the end of a run, the code writes four CSV files:

- `centerline_u_sim.csv`;
- `centerline_u_ghia.csv`;
- `centerline_v_sim.csv`;
- `centerline_v_ghia.csv`.

The reference data are the classical lid-driven-cavity values of Ghia, Ghia, and Shin at $\mathrm{Re} = 100$ [1]. The project exports both numerical and reference profiles, but does not currently compute an automated interpolation-based error norm between them.

13.4 Flow diagnostics

After every step the program writes the domain means of the kinetic energy, the enstrophy and the palinstrophy,

$$E = \tfrac{1}{2}\langle u^2 + v^2\rangle, \qquad Z = \tfrac{1}{2}\langle \omega^2\rangle, \qquad P = \tfrac{1}{2}\langle |\nabla\omega|^2\rangle, \tag{53}$$

to `output/integrals.csv`, with $\nabla\omega$ from D_x, D_y. The option `-integrals-interval` writes them less often. On a periodic grid $\langle\cdot\rangle$ is the mean over the nodes; with walls it is the trapezoidal mean over the domain, with the wall velocities on the wall nodes. The GPU computes them by reductions on the device, so no field is copied to the host for them. In an unforced periodic flow the continuum budgets are $\mathrm{d}E/\mathrm{d}t = -2\nu Z$ and $\mathrm{d}Z/\mathrm{d}t = -2\nu P$.

On a periodic grid, each VTK frame is accompanied by `output/spectrum-1-<n>.csv` with spectra summed over shells of $|\mathbf{k}|$ (binned, not averaged over the modes of a shell) and the spectral fluxes. Mode $\mathbf{k} = (2\pi m/L_x, 2\pi n/L_y)$ falls in shell b when $|b\,\Delta k - |\mathbf{k}|| \le \Delta k/2$, $\Delta k = 2\pi/\max(L_x, L_y)$, and

$$E(k_b) = \tfrac{1}{2}\sum_{\mathbf{k}\in b}\left(|\hat{u}_{\mathbf{k}}|^2 + |\hat{v}_{\mathbf{k}}|^2\right), \qquad Z(k_b) = \tfrac{1}{2}\sum_{\mathbf{k}\in b}|\hat{\omega}_{\mathbf{k}}|^2, \tag{54}$$

with the transforms normalised so that $\sum_b E(k_b) = E$ and $\sum_b Z(k_b) = Z$; a spectral density is $E(k_b)/\Delta k$, which has the same slopes. Let $A(\mathbf{k}) = |\tilde{D}_x|^2 + |\tilde{D}_y|^2$ from the symbols of the first-derivative operators that give u and v, and $Q(\mathbf{k}) = -(\tilde{D}_{xx} + \tilde{D}_{yy})$ from the Laplacian, so that $\hat{\omega} = Q\hat{\psi}$ and the energy of a mode is $\tfrac{1}{2}A|\hat{\psi}|^2$. The two differ from $|\mathbf{k}|^2$ by different amounts ($D_{xx} \neq D_x^2$), most near the Nyquist wavenumber. The fluxes through k_b are

$$\Pi_E(k_b) = -\sum_{b'\le b}\sum_{\mathbf{k}\in b'}\frac{A}{Q}\,\mathrm{Re}\left(\hat{\psi}^*_{\mathbf{k}}\hat{N}_{\mathbf{k}}\right), \qquad \Pi_Z(k_b) = -\sum_{b'\le b}\sum_{\mathbf{k}\in b'}\mathrm{Re}\left(\hat{\omega}^*_{\mathbf{k}}\hat{N}_{\mathbf{k}}\right), \tag{55}$$

where $N = -(uD_x\omega + vD_y\omega)$, or N_s of eq. (23), is the solver's own nonlinear term and $\hat{\psi}$ solves $(D_{xx} + D_{yy})\psi = -\omega$ with the solver's eigenvalues (section 8.7). The factor A/Q makes Π_E the transfer of the same energy that $E(k)$ measures: summed over all shells, the transfers equal the rates at which N changes $\frac{1}{2}\langle u^2 + v^2\rangle$ and $\frac{1}{2}\langle\omega^2\rangle$ in the discrete equations, to round-off. Without it ($\mathrm{Re}(\hat{\psi}^*\hat{N})$, the continuum expression) the energy transfer is off by 0.5–5 % on a random field on 16^2–24^2 grids. A positive flux carries energy or enstrophy towards larger k. The files also hold the viscous dissipation of each shell as the discrete equations have it: the viscous term $\nu(D_{xx} + D_{yy})\omega$ has symbol $-\nu Q$, so it removes

$$D_E(k_b) = \nu \sum_{\mathbf{k}\in b} \frac{A}{Q}|\hat{\omega}_\mathbf{k}|^2, \qquad D_Z(k_b) = \nu \sum_{\mathbf{k}\in b} Q|\hat{\omega}_\mathbf{k}|^2 \tag{56}$$

of energy and enstrophy, against the continuum $2\nu Z(k_b)$ and $2\nu P(k_b)$, which they approach where $A/Q \to 1$. Drag removes exactly $2\alpha E$. In general a damping term of symbol $-\sigma(\mathbf{k})$ removes $\sigma(A/Q^2)|\hat{\omega}|^2$ of energy and $\sigma|\hat{\omega}|^2$ of enstrophy per mode. D_E and D_Z hold the small-scale terms, viscosity ($\sigma = \nu Q$) and hyperviscosity ($\nu_h Q^p$); F_E and F_Z hold the large-scale ones, drag ($\sigma = \alpha$, so $\sum F_E = 2\alpha E$) and hypodrag (α_h/Q). The energy budget of the discrete equations is therefore $\mathrm{d}E/\mathrm{d}t = \sum_b T_E(k_b) - \sum_b (D_E + F_E)(k_b) + I_\mathrm{disc}$ exactly, and the enstrophy budget likewise without forcing. The option `-spectrum-interval` writes the spectra more often than the VTK frames, for time averages. In the continuum the nonlinear term conserves both, so Π_E and Π_Z vanish beyond the largest shell; in the discrete equations they do so up to the order of the derivatives on resolved fields (section 14.9), and Π_Z exactly with the skew-symmetric form.

14 Verification and Validation

It is useful to separate *verification* from *validation*. Verification asks whether the implementation solves its stated discrete equations correctly. Validation asks whether those equations reproduce accepted physical/reference results to the required accuracy.

14.1 Analytic verification of the FFT Poisson solver

The test suite constructs a discrete sine eigenmode

$$f_{ij} = \sin\left(\frac{\pi p\, i}{n-1}\right)\sin\left(\frac{\pi q\, j}{n-1}\right) \tag{57}$$

on the nodes $i, j = 0, \ldots, n-1$ (it vanishes on the wall nodes), with $n = 32$, $p = 3$, and $q = 5$. Since this is an exact eigenvector of the discrete Laplacian, the exact discrete solution is

$$\psi = \frac{f}{\lambda_{p,q}}. \tag{58}$$

The test requires the maximum relative difference to be no larger than 10^{-10}. This directly verifies the transform, eigenvalue division, and normalization.

14.2 Residual verification of the iterative Poisson solvers

For random right-hand-side data on a 24×24 grid, Gauss–Seidel and SOR are run with a tight iterate-change tolerance of 10^{-9}. The test then evaluates the actual five-point residual

$$r_{ij} = (L_h\psi)_{ij} - f_{ij} \tag{59}$$

and requires

$$\|r\|_\infty \le 10^{-4}. \tag{60}$$

This is valuable because the production stopping criterion is based on successive iterate change rather than directly on r.

14.3 Short cavity integration tests

Both Euler and RK4 are exercised for 50 timesteps on a 32×32 lid-driven cavity at $\mathrm{Re} = 100$, with sixth-order interior derivative operators, the FFT Poisson solver, and $\Delta t = 0.002$. The tests require:

- finite fields with bounded vorticity magnitude;
- a nontrivial velocity response to the moving lid;
- maximum discrete divergence no larger than 10^{-9}.

These are integration/smoke tests rather than accuracy tests against a fully converged reference solution.

14.4 CPU–GPU equivalence tests

When built with CUDA and run on a usable GPU, the test suite checks:

- host/device field round trips;
- continuity reductions on grids up to 300×300;
- all four CSR derivative operators on several grid sizes;
- FFT, SOR, and Gauss–Seidel Poisson solutions;
- Euler and RK4 full timesteps;
- multiple grid sizes, including 128×128;
- a case with four independently moving walls.

Typical comparison thresholds in the tests are 10^{-12} for CSR and FFT building blocks, 10^{-11} for full FFT-based timesteps, and 10^{-7} for converged iterative-solver comparisons. With OpenMP enabled, additional tests compare CPU and GPU red–black iterative solvers at the shipped production tolerance.

A CUDA test build with no usable GPU returns exit status 77 rather than reporting a false pass.

14.5 Spatial convergence against a manufactured solution

The cavity cannot measure the spatial order of the method: the lid is discontinuous at the top corners, the vorticity is singular there, and refinement on the cavity measures the singularity. The Ghia data are themselves a second-order solution on a 129×129 grid. The order is therefore measured against a manufactured solution,

$$\psi(x, y, t) = g(t)\, \sin^2\!\left(\frac{\pi x}{L_x}\right) \sin^2\!\left(\frac{\pi y}{L_y}\right), \qquad g(t) = 0.2 + 0.1 \sin(2\pi t). \tag{61}$$

It has $\psi = 0$ and $u = v = 0$ on every wall, so it uses the solver's ordinary stationary no-slip walls and the $\psi = 0$ Poisson condition without any change. It is made an exact solution by an optional source term f in the vorticity equation (the curl of a body force, were it written in the momentum equation),

$$\frac{\partial \omega}{\partial t} = -u\frac{\partial \omega}{\partial x} - v\frac{\partial \omega}{\partial y} + \frac{1}{\mathrm{Re}}\nabla^2 \omega + f, \qquad f = \frac{\partial \omega_e}{\partial t} + u_e\frac{\partial \omega_e}{\partial x} + v_e\frac{\partial \omega_e}{\partial y} - \frac{1}{\mathrm{Re}}\nabla^2 \omega_e, \tag{62}$$

where the subscript e denotes the exact fields from eq. (61), all in closed form. The source is the `vorticity_source` callback of `solver_config`; it is unset in normal runs. The solver evaluates it at the time of each right-hand-side evaluation, so at t, $t + \Delta t/2$, $t + \Delta t/2$ and $t + \Delta t$ in RK4, with $t = t_0 + n\,\Delta t$ after n steps of a run that starts at `t0`, and the CUDA backend fills it on the host and copies it to the device.

Each run starts from the exact state at $t = 0$, uses RK4 and the FFT solver at $\mathrm{Re} = 100$, and is compared with the exact solution at $t = 0.25$. The time step is at most 2.5×10^{-3} and at most half the viscous stability limit; a run with a ten to thirty times smaller step gives the same errors to all digits shown, so the time error is negligible. The observed order between two grids is

$$p_{\mathrm{obs}} = \log_2 \frac{\|e_h\|_\infty}{\|e_{h/2}\|_\infty}. \tag{63}$$

For derivative order 6 on the unit square:

Grid	ψ	u	ω, walls included	ω, interior
33^2	1.02×10^{-3}	2.97×10^{-3}	2.89×10^{-2}	2.01×10^{-2}
65^2	2.49×10^{-4} (2.03)	7.44×10^{-4} (2.00)	3.84×10^{-3} (2.91)	3.26×10^{-3} (2.62)
129^2	6.12×10^{-5} (2.02)	1.86×10^{-4} (2.00)	6.19×10^{-4} (2.63)	5.50×10^{-4} (2.57)
257^2	1.52×10^{-5} (2.01)	4.67×10^{-5} (2.00)	1.19×10^{-4} (2.37)	1.11×10^{-4} (2.31)
513^2	3.78×10^{-6} (2.01)	1.17×10^{-5} (2.00)	2.58×10^{-5} (2.21)	2.47×10^{-5} (2.16)

(maximum errors over the nodes, observed order in parentheses). The findings:

- For this smooth solution with stationary no-slip walls, the complete method converges at second order in space with every derivative order. ψ, u and v converge at 2.0 on every grid. ω converges faster on the coarse grids and approaches second order under refinement; its largest error is at the walls.
- Orders 4 and 6 give the same errors to three digits, so the sixth-order rows of the deep interior do not affect the result. The order is held back by something all derivative orders share: the five-point Poisson operator (eq. (31)) and the low-order rows next to the walls (table 1). Section 14.6 separates the two.
- Compared with order 2, orders 4 and 6 reduce the vorticity error by about 2.6× at 513^2 (6.9×10^{-5} to 2.6×10^{-5}, walls included) and leave the velocity error essentially unchanged (1.02×10^{-5} for u with order 2, 1.17×10^{-5} with order 6).
- A 2×1 domain and a unit-square grid with $\Delta x = \Delta y/2$ show the same orders.

`make test` runs a short version: on 17^2, 33^2 and 65^2 it requires the observed orders of ψ, u and v between 33^2 and 65^2 to be within 0.15 of 2 and that of ω to be at least 1.9, and it bounds the errors at 65^2. It also checks that Euler and RK4 keep their first and fourth order in time with the time-dependent source, which fails if an RK4 stage evaluates the source at the wrong time, and that a run started at $t_0 = 0.4$ is as accurate as one started at $t = 0$. `make convergence` runs the full study up to 257^2 for orders 2, 4 and 6 and the two rectangular cases.

14.6 What limits the spatial order: an ablation

To find which part of the discretization holds the order to two, a copy of the RK4 step (with nothing replaced, it computes exactly what `step()` computes, bit for bit) replaces parts of the method with the exact solution at each stage's time:

- *exact wall* ω: the wall vorticity is exact; the Poisson solve and the velocity $u = D_y\psi$, $v = -D_x\psi$ are as usual;

- *exact ψ*: ψ is exact, so the Poisson operator plays no part; u, v and the wall vorticity $D_x v - D_y u$ are computed from it;
- *exact u, v*: the velocity is exact at every node; the wall vorticity is computed from it.

Combinations replace several parts. With exact u, v and exact wall ω, only the transport derivatives of ω remain. For derivative order 6, maximum error of ω over all nodes at 257^2, with the observed order from 129^2:

Replaced by the exact solution	ω error	order of ω	order of u
nothing (the solver)	1.2×10^{-4}	2.37	2.00
wall ω	5.1×10^{-5}	2.00	2.00
ψ	1.2×10^{-3}	2.00	3.00
ψ and wall ω	2.8×10^{-7}	4.02	3.00
u, v	5.9×10^{-4}	2.00	–
u, v and wall ω	2.7×10^{-7}	3.89	–

Order 4 gives the same orders (measured up to 129^2); order 2 gives 2.00 in every variant.

- **Both the Poisson operator and the wall-vorticity closure limit the order to two, each on its own.** With the exact wall vorticity, the five-point Poisson operator still gives second order; with the exact ψ, the wall closure still gives second order. Replacing ψ by the exact solution also removes the coupling between ψ and the wall formula, which a real operator keeps; section 14.7 measures the real higher-order operators.
- **The two errors partly cancel.** Replacing only ψ by the exact solution makes the vorticity error ten times *larger* (1.2×10^{-3} against 1.2×10^{-4}), although the solver's error contains the wall closure's error as well as the Poisson operator's: in the coupled scheme the two partly cancel. This is consistent with the observation that a locally low-order wall formula such as Thom's can be part of a globally second-order method.
- **With both replaced, the order rises to four, not six.** With the exact velocity and wall vorticity, only the derivative operators are left, and ω converges at 3.9, approaching four. For this solution, that is consistent with the operators' rows next to the walls, which are of second and fourth order (table 1); the first- and second-derivative rows were not ablated separately. Without walls the same stencils reach sixth order (section 14.8).

`make ablation` prints this study for orders 2, 4 and 6 up to 129^2; `make test` checks three of its results on 33^2 and 65^2 (orders 2, 2 and 4 for exact wall ω, exact ψ, and both).

14.7 Raising the order with walls

The compact Poisson operator (section 8.6) and the third-order wall closure (eq. (20)) replace the two parts the ablation identified. With derivative order 6, on the manufactured solution of section 14.5, maximum errors at 257^2 and the observed order from 129^2:

Poisson operator, wall closure	ω error	order of ω	order of ψ	order of u
five-point, velocity-based (default)	1.19×10^{-4}	2.37	2.01	2.00
five-point, eq. (20)	9.65×10^{-5}	2.05	2.00	2.00
compact, velocity-based	7.52×10^{-5}	3.01	2.99	2.79
compact, eq. (20)	3.16×10^{-7}	4.20	4.21	3.00
and fourth-order velocity rows	3.06×10^{-7}	4.10	4.21	4.92

Derivative order 4 gives the same orders; order 2 stays at two.

- **Together they give fourth order.** ω converges at 4.0 on the interior and at 4.2 including the walls, ψ at 4.2, and the error at 257^2 is 380 times smaller than the default's.
- **The wall closure alone changes nothing**: the five-point Poisson operator keeps the order at two.
- **The compact operator alone gives three**, more than the ablation's exact-ψ variant suggested: with the real operator, ψ and the velocity-based wall formula stay coupled, and the result is one order above what the formula's local accuracy suggests, as is known for Thom's formula.
- **With these two, the velocity converges at three.** $u = D_y\psi$ uses the first-derivative rows next to the walls (table 1), which limit it; ω is not limited by them here.
- **Fourth-order velocity rows lift the velocity to fourth order.** With `-velocity-order 4` (`solver_config.DXv`, `DYv`), $u = D_y\psi$ and $v = -D_x\psi$ use a first-derivative operator whose rows at the wall and next to it are fourth-order one-sided, $(-25f_0 + 48f_1 - 36f_2 + 16f_3 - 3f_4)/(12h)$ and $(-3f_0 - 10f_1 + 18f_2 - 6f_3 + f_4)/(12h)$, and whose other rows are those of `SDiff1` (`SDiff1_wall4`). Only the velocity and the continuity check use it, so the divergence of the velocity stays at round-off and the transport is unchanged. With the other two options, u and v then converge at fourth order: the new rows are formally fourth order, derivative order 4 measures 4.0, and derivative order 6 measures 4.9 over the finest grids for this solution, presumably a favourable error structure rather than a higher order; and the error of u at 257^2 falls from 2.3×10^{-6} to 1.8×10^{-9}. On its own, without the compact Poisson operator, it leaves the order at two.

All three options are stable at the same time step: the default cavity runs bounded for 3000 steps at 0.95 times the stability limit with RK4 and with Euler. The CPU and GPU solvers give the same result to 10^{-15}. `make test` checks the compact operator's order, the pairing (order at least 3.7 from 33^2 to 65^2, and two for the wall closure alone) and the GPU against the CPU; `make convergence` prints the four combinations.

The cavity. The corners of the lid make the cavity singular, so its order is not that of the manufactured solution, but the two options still converge much faster. For the default case (Re = 100, $t = 30$), the largest difference of the centerline velocities at the 15 interior Ghia points from the 257^2 solution with both options is 3.2×10^{-3} at 64^2 and 5.6×10^{-4} at 129^2 with the defaults, against 4.8×10^{-4} and 1.3×10^{-5} with both options (5.0×10^{-4} and 1.5×10^{-5} with the velocity rows as well: the cavity's singular corners, not the velocity, limit it). The two 257^2 solutions agree to 1.0×10^{-4}, and both lie 0.005 (u) and 0.009 (v) from Ghia et al. Measured against Ghia et al., the 64^2 solution with both options looks worse than the default (0.0055/0.0090 against 0.0023/0.0073): the default's small distance at 64^2 comes partly from error cancellation (section 14.12), and the reference itself is a second-order solution on 129^2. The defaults are unchanged; both options are opt-in.

14.8 Spatial order without walls

On the periodic grid of section 8.7, the manufactured solution is a sum of three Fourier modes,

$$\psi = g(t) \sum_{m=1}^{3} A_m \cos\left(2\pi \left(\frac{a_m x}{L_x} + \frac{b_m y}{L_y}\right) + \varphi_m\right), \tag{64}$$

with $(a_m, b_m) = (1,1), (2,-1), (1,3)$ and amplitudes $(1, \frac{1}{2}, \frac{1}{4})/(2\pi)$. The wavenumbers differ, so the nonlinear term does not vanish. With the same $g(t)$, Re, final time and time-step control as in section 14.5, the maximum error of ω is

Grid	order 2	order 4	order 6
32^2	3.34×10^{-1}	1.87×10^{-2}	1.23×10^{-3}
64^2	8.44×10^{-2} (1.99)	1.22×10^{-3} (3.94)	2.05×10^{-5} (5.91)
128^2	2.11×10^{-2} (2.00)	7.68×10^{-5} (3.98)	3.25×10^{-7} (5.98)
256^2	5.27×10^{-3} (2.00)	4.81×10^{-6} (4.00)	5.09×10^{-9} (6.00)

and ψ, u and v show the same orders (ψ reaches 1.9×10^{-11} at 256^2 with order 6). Without walls, and with a Poisson operator of the same order, the method reaches the nominal order of its stencils, which completes the picture of section 14.6: the second order of the wall-bounded method comes from the wall closures and the five-point Poisson operator, not from the interior stencils. `make test` checks the orders between 32^2 and 64^2 (within 0.15 of 2, 4 and 6) and on a 2×1 domain; `make regression` checks the Taylor–Green vortex, whose error falls by a factor of 64 from 32^2 to 64^2.

Compact schemes. The compact operators of section 8.8 act on every Fourier mode of 16-, 33- and 128-point grids as their symbols to 4×10^{-14}; they annihilate constants and the first derivative is antisymmetric to round-off. On the periodic manufactured solution they converge at order 6.04 (ω) and 6.01 (ψ) from 32^2 to 64^2, with an error at 64^2 twelve times smaller than explicit order 6 (1.6×10^{-6} against 2.0×10^{-5}). On the Taylor–Green vortex (64^2, $t = 0.5$) the error is six times smaller (1.1×10^{-10} against 6.3×10^{-10} of $\max|\omega|$). The GPU matches the CPU with them to 3×10^{-15}.

14.9 Diagnostics

For the Taylor–Green vortex the integrals are $E = \frac{1}{4}e^{-4\nu k^2 t}$, $Z = 2k^2E$ and $P = 4k^4E$ with $k = 2\pi$, and all energy lies in the shell $|\mathbf{k}| = \sqrt{2}\,k$. On 32^2 (order 6) the computed E and Z at $t = 0$ are exact to round-off, P is within 8×10^{-7} (it uses D_x, D_y), E at $t = 0.1$ is within 6×10^{-7} of the exact decay, and the spectrum holds all energy in that shell to 10^{-14}. On the unforced three-mode periodic flow of section 14.8 at $\mathrm{Re} = 100$:

Order, grids	energy transfer	enstrophy transfer	$\lvert\dot{E} + 2\nu Z\rvert / 2\nu Z$
2, 64^2 / 128^2	9.8×10^{-3} / 2.5×10^{-3}	1.3×10^{-2} / 3.3×10^{-3}	1.1×10^{-2} / 2.8×10^{-3}
4, 64^2 / 128^2	6.8×10^{-4} / 4.3×10^{-5}	6.3×10^{-4} / 4.0×10^{-5}	1.7×10^{-4} / 1.1×10^{-5}
6, 64^2 / 128^2	2.8×10^{-5} / 4.5×10^{-7}	2.5×10^{-5} / 4.0×10^{-7}	3.0×10^{-6} / 7.2×10^{-8}

(net nonlinear transfers, relative to the largest flux). The spectra sum to the integrals to round-off. The nonlinear term conserves energy and enstrophy, and the energy budget closes, up to errors that fall at the order of the derivatives. On a pseudo-random field the summed spectral transfers equal the physical-space nonlinear rates of E and Z to 3×10^{-15} (orders 2 and 6, 16^2 and 24×20). On a random field on 24^2, where A/Q is far from 1 over much of the spectrum, the centred difference of E over two RK4 steps matches $\sum T_E - \sum D_E$ to 6×10^{-7} (order 2) and 1.4×10^{-6} (order 6), the time-stepping error, while $\mathrm{d}E/\mathrm{d}t = -2\nu Z$ is off by 180 % and 75 %: on fields resolved up to the grid cutoff only the discrete budget closes. `make test` checks these, and the GPU integrals against the CPU; `make regression` checks the Taylor–Green energy written to `integrals.csv`.

14.10 Forcing and drag

Kolmogorov forcing ($A = 1$, $n = 2$, $\nu = 0.05$, $\alpha = 0.1$, 32^2, order 6) reaches the discrete laminar state $f_K/(\nu Q + \alpha)$, with Q the symbol of D_{yy}, to 4×10^{-11}, and the continuum one to 6×10^{-6}; the physical input I balances $2\nu Z + 2\alpha E$ to 2.5×10^{-5}, the order of the scheme, and the discrete input I_{disc} balances $\sum D_E + 2\alpha E$ to 4×10^{-11}, the convergence of the steady

state ($I_{\mathrm{disc}}/I = 0.999981$). Drag on the Taylor–Green vortex gives $E = \frac{1}{4}e^{-(4\nu k^2+2\alpha)t}$ to 6×10^{-7}. A random kick carries $\varepsilon\,\Delta t$ of discrete energy to 5×10^{-15}, and two generators with the same seed make the same kick. The GPU matches the CPU to 3×10^{-15} with RK4 and Euler, with every term at once: drag, Kolmogorov and random forcing, hyperviscosity, hypodrag and the skew-symmetric nonlinear term. With drag, hypodrag and hyperviscosity of order 2 and 3 on a random field (24^2, order 4), the energy and the enstrophy budgets $\mathrm{d}E/\mathrm{d}t = \sum T_E - \sum(D_E + F_E)$ and $\mathrm{d}Z/\mathrm{d}t = \sum T_Z - \sum(D_Z + F_Z)$ close to 4×10^{-7} and 10^{-6}, the time-stepping error (both fall by four when Δt is halved). On a random 24×20 field the net enstrophy transfer of the skew-symmetric form is below 3×10^{-16} of the largest shell transfer for orders 2, 4 and 6, against 0.4–0.7 for the advective form, and the skew-symmetric form converges at orders 2.00, 3.97 and 5.95 on the manufactured solution (64^2 to 128^2; on 32^2 to 64^2, 3.85 and 5.72 for orders 4 and 6, as the products $u\omega$, $v\omega$ carry twice the wavenumbers). `make regression` checks the laminar Kolmogorov energy and energy input from the command line.

14.11 Two-dimensional turbulence

Forced two-dimensional turbulence has two inertial ranges [6]: energy cascades to scales larger than the forcing scale, with $E(k) \propto \varepsilon^{2/3}k^{-5/3}$ and $\Pi_E = -\varepsilon$, and enstrophy to smaller scales, with $E(k) \propto \eta^{2/3}k^{-3}$ and $\Pi_Z = \eta$. Ranges this clean need the dissipation confined to the two ends of the spectrum. Plain viscosity and linear drag act at every scale: a first 256^2 run with them ($\mathrm{Re} = 9000$, $\alpha = 0.1$) gave the right signs of both fluxes but no plateau. The runs below use hyperviscosity ($p = 4$) and hypodrag (section 8.5) and no viscosity to speak of ($\mathrm{Re} = 10^7$). All are on 512^2 with order 6 and RK4, on the GPU. Each injects $\varepsilon = 0.1$ with the random forcing on the shell $|K - K_f| \le 1$, with $K = |\mathbf{k}|/2\pi$, and writes the spectra every 0.25 (direct) or 0.1 (inverse) time units; `tools/cascade.py` averages them.

Run	K_f	ν_h	α_h	Δt	steps	mean over
direct	4	6.5×10^{-24}	40	5×10^{-4}	60 000	$t = 10$–30
inverse	40	1.6×10^{-23}	1200	4×10^{-4}	30 000	$t = 4$–12

Reproducing the runs. The two forced runs and the decaying run of this section, on 512^2 on the GPU, and the scripts that average and plot them:

```
# direct cascade: forced at |k|/2pi = 4, 60 000 steps (25 min on an RTX 3050 laptop
    GPU)
./cnavier --case forced --n 512 --re 1e7 --drag 0 --hypodrag 40 --hyperviscosity 6.5e
    -24 \
    --hyper-order 4 --advection skew --forcing-rate 0.1 --forcing-k 4 --dt 5e-4 --tf
    30 \
    --output-interval 20000 --spectrum-interval 500 --integrals-interval 100
python3 tools/cascade.py plot output --kf 4 --from 10
python3 tools/cascade.py check output --kf 4 --from 10 --below 1,2 --above 10,100 \
    --conserves-enstrophy --min-pi-z 40 --spread-z 0.1
python3 tools/cascade.py budget output --from 10 --eps 0.1 --kf 4

# inverse cascade: forced at |k|/2pi = 40
./cnavier --case forced --n 512 --re 1e7 --drag 0 --hypodrag 1200 --hyperviscosity 1.6
    e-23 \
    --hyper-order 4 --advection skew --forcing-rate 0.1 --forcing-k 40 --dt 4e-4 --tf
    12 \
    --output-interval 10000 --spectrum-interval 250 --integrals-interval 50

# decaying turbulence
./cnavier --case decaying --n 512 --re 1e7 --hyperviscosity 1e-23 --hyper-order 4 \
```

```
    --advection skew --dt 4e-4 --tf 20 --output-interval 12500 --spectrum-interval
    1250 \
    --integrals-interval 50
python3 tools/cascade.py evolution output --times 0,1,5,20
python3 tools/snapshots.py output 0 1 2 3 --titles "t = 0,t = 5,t = 10,t = 15"
```

`make regression` runs a 64^2 version of the direct-cascade check (about 6 s on the CPU): the signs of Π_E below and Π_Z above the forcing, and the conservation of enstrophy.

Direct cascade. With the skew-symmetric form (eq. (23)), $\Pi_Z = 43.8$ on average over $K = 10$–100 and stays within 42.7–46.3 there, a plateau over a decade (fig. 2). Above the forcing $|\Pi_E| \le 4 \times 10^{-4}$, 0.4 % of ε; below it $\Pi_E < 0$. The spectrum falls as $k^{-3.3}$ (fit over $K = 8$–100), slightly steeper than k^{-3}, as is usual at this range. The net nonlinear enstrophy transfer, $-\Pi_Z$ through the last shell, is -3×10^{-15}. Of the enstrophy input, 44 is removed by the hyperviscosity and 17 by the hypodrag; of the energy, 0.090 by the hypodrag and 6×10^{-5} by the hyperviscosity. The energy accounting over the window (`tools/cascade.py budget`, from a rerun that writes the spectra every 100 steps and reproduces the frames above bitwise) separates drift from noise. E goes from 0.213 to 0.204, a drift of -4×10^{-4} per unit time, so there is no secular trend to speak of. The dissipation removes 0.0904 ± 0.003 (block-averaged standard error), and the net nonlinear transfer is -7×10^{-6}. The input these imply, $\mathrm{d}E/\mathrm{d}t + \sum D_E + \sum F_E - \sum T_E = 0.090$, is 10 % below ε. But ε is only the *expected* input. The realized input of the kicks over a window T_w differs from it by the accumulated cross terms $\langle \mathbf{u} \cdot \delta\mathbf{u} \rangle$, whose standard deviation is about $\sqrt{2\varepsilon E_f / (M T_w)}$, with E_f the energy in the M forced modes. The variance of one kick's cross term is $2E_f \varepsilon \Delta t / M$, and the kicks are independent. With only $M = 28$ forced modes this is 0.006 here, the largest term in a combined standard error of 0.0065, so the deficit is 1.5 standard errors. Four 128^2 runs of the same set-up with other seeds ($\nu_h = 4.2 \times 10^{-19}$ for the coarser grid, averaged over the longer window $t = 10$–60) close the budget to −0.4, +1.7, +0.8 and +0.6 %, each within 0.3 of its combined standard error. The kick accounting is therefore unbiased. The 10 % of the 512^2 window is the scatter of a 20-unit window with 28 forced modes. The scatter between the four seeds, 0.0009, is below the estimate 0.004, which counts all the energy of shells 3–5 as forced and is an upper bound. With 400 frames the plateau is 42.4–44.7 (mean 43.7). With the advective form the same run has a less flat flux (45.5–51.3), and near the cutoff the nonlinear term *creates* enstrophy at 14.3 per unit time, 23 % of the input, so that Π_Z ends at −14 instead of 0.

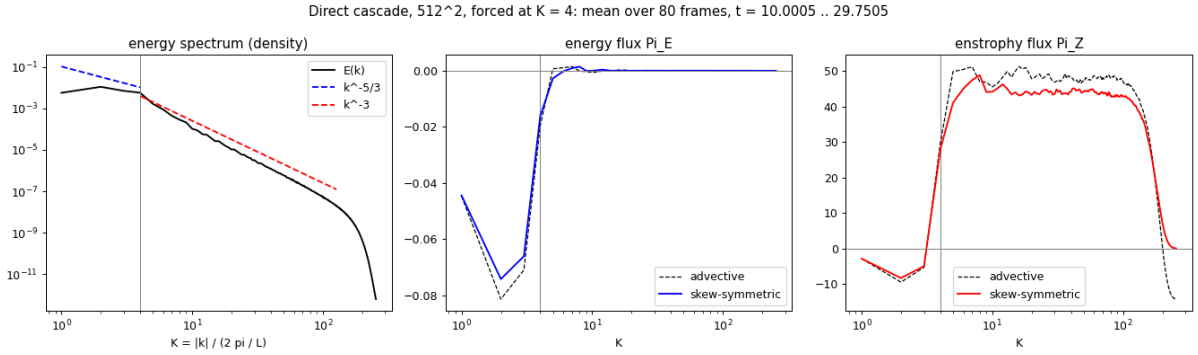


Figure 2: Direct cascade: time-mean energy spectrum (with $k^{-5/3}$ and k^{-3} for reference) and fluxes, skew-symmetric form; dashed, the advective form.

Inverse cascade. Π_E lies between −0.093 and −0.096, about $-0.95\,\varepsilon$, over $K = 20$–38 (fig. 3), and $k^{5/3}E(k)$ is flat to ±6 % over $K = 12$–30. The hypodrag, whose rate is α_h/k^2, takes the

energy out over $K \approx 2$–15, which ends the plateau there. The hypodrag removes 0.093 and the hyperviscosity 0.004, and the nonlinear term 0.001, 3 % short of $\varepsilon = 0.1$. The difference is a secular drift, not noise: E still rises from 0.152 to 0.170 over the window, $\mathrm{d}E/\mathrm{d}t = +0.0023$, so the state is close to stationary but not exactly. With the drift the implied input is 0.1005, within 0.5 % of ε (0.7 standard errors). With $M = 248$ forced modes the realized input scatters by only 0.0006, unlike the direct run. Over 400 frames Π_E stays within -0.0912 to -0.0951 on $K = 20$–38 (spread 4 %). Above the forcing the enstrophy flux is 5.8–5.9×10^3 over $K = 42$–100, and the net enstrophy transfer is 10^{-13}. With the advective form the nonlinear term creates 2.85×10^3 per unit time near the cutoff, 45 % of the input; the hyperviscosity then removes 8.8×10^3 instead of 5.9×10^3. The implied Kolmogorov constant, $k^{5/3}E/|\Pi_E|^{2/3} \approx 9$, is above the ≈ 6 of well-resolved studies [8], which is not surprising for an inertial range of a factor of 2.5 squeezed between the forcing and the hypodrag.

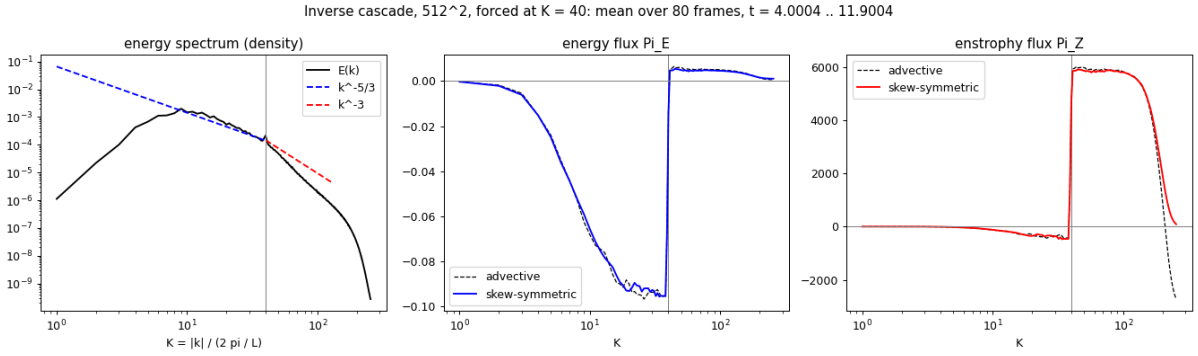


Figure 3: Inverse cascade: as fig. 2, forced at $K = 40$.

Decaying turbulence. `-case decaying` starts from random phases with modal amplitudes of envelope $E(k) \propto (k/k_0)^4 e^{-2(k/k_0)^2}$ and $E = \frac{1}{2}$ (section 8.7). On 512^2 with $k_0 = 10$, the hyperviscosity of the direct run ($\nu_h = 10^{-23}$, $p = 4$), $\mathrm{Re} = 10^7$ and the skew-symmetric form, run to $t = 20$ (50 000 steps), E falls by 0.6 % while Z falls by a factor of 15, as it should at high Reynolds number: the direct cascade carries enstrophy to the cutoff, where it is dissipated, and energy goes to large scales, where nothing removes it (fig. 4). The peak of the spectrum moves from $K = 9$ to $K = 1$, which holds 59 % of the energy at the end. Isolated coherent vortices emerge and merge [7], and Z drops in steps at the mergers.

Resolution. How much of the spectrum does each order resolve? Decaying turbulence at $\mathrm{Re} = 2 \times 10^4$ (skew-symmetric form, $k_0 = 10$) was run on 256^2 and 512^2 with orders 2, 4 and 6 and the compact scheme of section 8.8, and on 2048^2 with order 6, all with the same time step (6.25×10^{-5}) and from the same initial field: the phases of the initial spectrum depend on the wavevector only, so every grid that resolves the spectrum gets the same field (`random_initial_field()`; `make test` checks it to 3×10^{-16}). Turbulence is chaotic, so the runs decorrelate. At $t = 0.5$ the shell energies differ by tens of per cent even at $K = 10$. At $t = 0.09$, about six eddy turnovers, after the cascade has filled the spectrum, the large scales ($K \le 20$) of the 512^2 runs still agree with the 2048^2 one to 2×10^{-4} (order 6) and 6×10^{-5} (compact). A common time step matters here: an earlier comparison with a reference whose frames fell half a step later agreed only to 2×10^{-3}. The spectra are compared there (fig. 8); the reference is resolved, with E at $K = 256$ at 10^{-8} of its peak, and order 6 on 2048^2 differentiates that shell to 0.2 %:

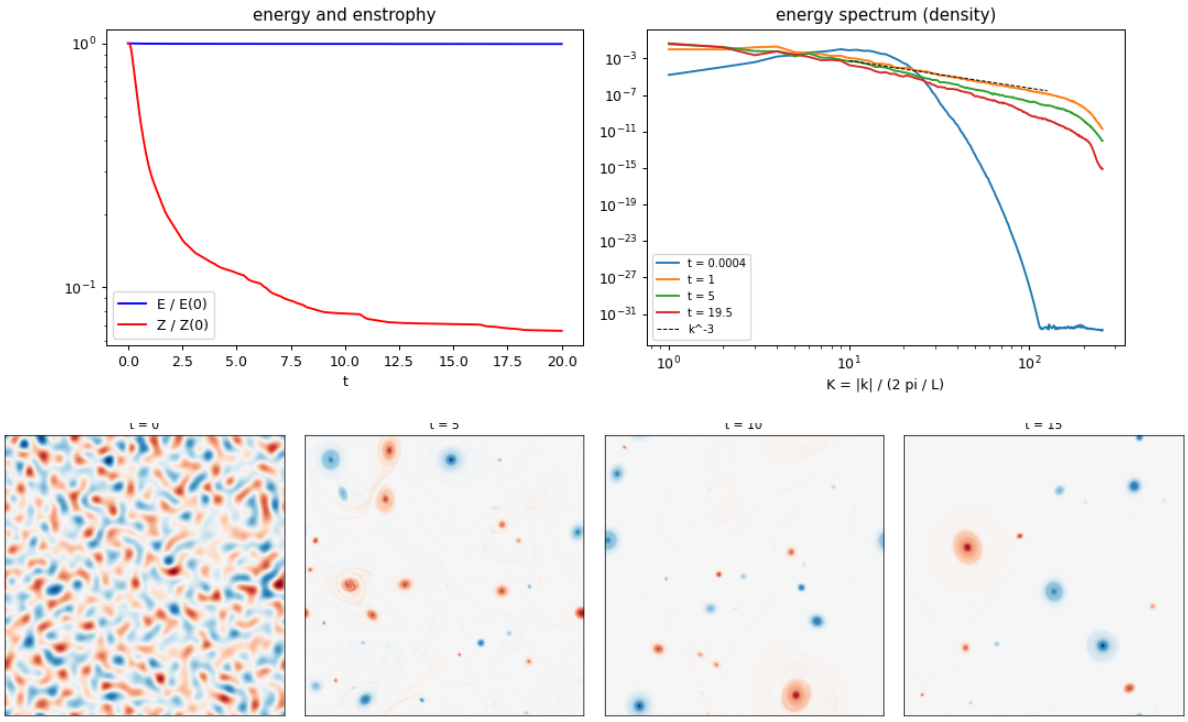


Figure 4: Decaying turbulence on 512^2: energy and enstrophy, energy spectra, and the vorticity at $t = 0$, 5, 10 and 15.

to 10 % (2 %) up to $K =$	spectral	spectral, 3/2	spectral, 2/3	compact 6	order 6	order 4	order 2
512^2 ($K_{\mathrm{Nyq}} = 255$)	235 (201)	239 (198)	143 (103)	198 (167)	152 (111)	133 (73)	60 (28)
256^2 ($K_{\mathrm{Nyq}} = 127$)	93 (57)	93 (61)	50 (29)	118 (67)	74 (48)	60 (30)	29 (1)

Orders 6, 4 and 2 resolve the spectrum to 10 % up to about 59, 49 and 23 % of the Nyquist wavenumber, and to 2 % up to 38–44, 24–29 and at most 11 %. The compact scheme resolves 78–93 % to 10 % and 53–65 % to 2 %: 1.3–1.6 times as many shells as explicit order 6 on the same grid, in line with its modified wavenumber (section 8.8). Beyond that the finite differences underestimate the derivatives, and the spectrum falls below the reference; the compact scheme on 256^2 first overshoots it, by up to 8 % around $0.85\,K_{\mathrm{Nyq}}$. Order 6 on 256^2 resolves more shells than order 2 on 512^2, at about half the cost per step (3.3 against 5.6 ms on the GPU). In its banded form the compact scheme costs 20 times explicit order 6 per step, so it does not pay; in Fourier space (section 8.9) it costs 1.3–1.4 times.

Pseudospectral differentiation (section 8.9) resolves the most on 512^2, where the spectrum has fallen to 10^{-8} of its peak at the Nyquist wavenumber: 92 % of the range to 10 % and 79 % to 2 %. On 256^2, where it is still at 10^{-6}, it resolves less than the compact scheme. Nothing damps the modes near the cutoff, so energy piles up there, up to twice the reference at the Nyquist wavenumber (fig. 5). Quadratic aliasing of the nonlinear term does not explain it: with 3/2 padding, which removes the quadratic aliasing from every mode but the Nyquist ones, the range is the same (239 and 93 shells) and so is the pile-up (1.8 and 2.2 times the reference). This is consistent with an accumulation caused by the truncation itself: the enstrophy the cascade carries to the cutoff is not dissipated fast enough before it, as in Galerkin-truncated flows [9]. These runs rule out quadratic aliasing of the nonlinear term as the explanation for the pile-up; they do not by themselves identify the mechanism. The finite differences underestimate the derivatives there instead, and their spectra fall below the reference. The 2/3 rule caps the range at 2/3 of the Nyquist wavenumber by construction, and the energy piles up below that cutoff, which

nothing dissipates either. It resolves 56 % (512^2) and 39 % (256^2) to 10 %, less than undealiased spectral and than compact. Its merit is exact conservation, not resolution.

What a resolved shell costs follows from the step time and the advective limit. Explicit time steps go as $h/(k^*h)_{\max}$: 1.59 for order 6, 1.99 compact, π spectral (padded or not), and $2\pi/3$ with the 2/3 rule. So the cost per unit simulated time at the advective limit is proportional to (ms per step)$\times(k^*h)_{\max}$. The resolved range of explicit order 6 grows linearly with n (74 on 256^2, 152 on 512^2), and its cost per unit time as n^3. To resolve on its own grid what compact resolves on 512^2 it needs $(198/152)^3 = 2.2$ times its 512^2 cost, 45 against the compact 35 (in ms per step times $(k^*h)_{\max}$): compact is 22 % cheaper. Pseudospectral is 26 % cheaper (56 against 75). On 256^2 compact is 2.4 times cheaper (9.5 against 23), and undealiased spectral is 34 % dearer (15.1 against 11.3). Padding costs 1.85 times per step (31.4 against 17.0 ms on 512^2, 7.8 against 4.7 on 256^2, five transforms on $(3n/2)^2$ points against three on n^2), for the same range: padded pseudospectral costs 1.3 (512^2) and 2.3 (256^2) times explicit order 6 at equal range. These gains are modest where the flow is resolved, and depend on how close to the cutoff the spectrum carries energy. In the tested decaying runs (one Reynolds number, one initial spectrum, two grids), compact sixth-order differentiation gave the most robust cost–resolution trade-off. Pseudospectral differentiation became advantageous once the grid resolved the high-wavenumber tail. The 2/3 rule trades a third of the range for exact conservation. The next paragraphs test whether this ordering holds at other Reynolds numbers, for other spectra and in forced turbulence.

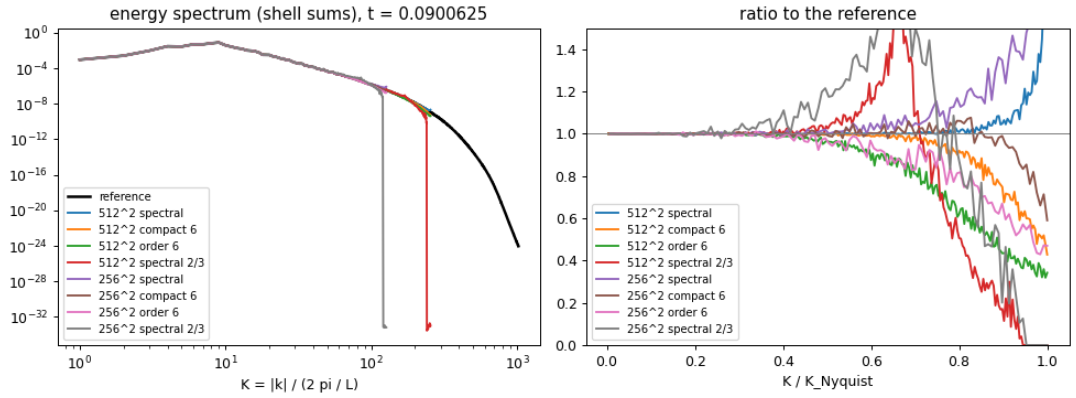


Figure 5: As fig. 8, with pseudospectral differentiation, with and without the 2/3 rule.

Does the ordering hold across regimes? The comparison above is one flow: decaying turbulence at $\mathrm{Re} = 2 \times 10^4$ from $k_0 = 10$. To see whether its ordering is robust, the same comparison was repeated varying one parameter at a time: $\mathrm{Re} = 5 \times 10^3$ and 8×10^4, and $k_0 = 5$ and 20, each on 256^2 and 512^2 against its own 2048^2 reference. The comparison time keeps t/τ_0 fixed, $\tau_0 \propto 1/k_0$ (all runs start with $E = \frac{1}{2}$): $t = 0.09 \cdot 10/k_0$, about six eddy turnovers. The largest scales ($K \le 20$) still agree with the reference to 10^{-3}–10^{-2} there, so the comparison is made before the runs decorrelate. A 2048^2 explicit-order-6 run is taken as the reference only where it does not depend on its own discretization. For the two most demanding cases ($\mathrm{Re} = 8 \times 10^4$, $k_0 = 20$) a second, independent 2048^2 run with the compact scheme agrees with it to 1.5×10^{-3} up to $K = 255$ (and to 10^{-4} up to $K = 127$), well inside the 2 % and 10 % tolerances measured; the two differ only beyond $K \approx 500$, the upper half of their own range. The question is whether a single parameter, the reference's energy at the grid's Nyquist shell relative to its peak, $E(K_{\mathrm{Nyq}})/E_{\max}$, orders the schemes. It measures how much of the flow lies at the cutoff. The scripts are `tools/compare_schemes.py` (per case and the plot of fig. 6).

case	grid	$E(K_{\mathrm{Nyq}})/E_{\max}$	order 6	compact 6	spectral	spectral, 3/2	spectral, 2/3
$\mathrm{Re} = 5 \times 10^3$	512^2	2.8×10^{-12}	144	196	**249**	248	163
$k_0 = 5$	512^2	4.2×10^{-11}	146	197	**246**	**246**	154
baseline	512^2	1.5×10^{-8}	152	198	235	**239**	143
$k_0 = 5$	256^2	9.0×10^{-8}	71	105	101	**109**	62
$\mathrm{Re} = 5 \times 10^3$	256^2	2.0×10^{-7}	74	105	**120**	**120**	70
$\mathrm{Re} = 8 \times 10^4$	512^2	2.3×10^{-7}	152	**217**	193	200	117
$k_0 = 20$	512^2	1.6×10^{-6}	159	213	**217**	**217**	130
baseline	256^2	4.9×10^{-6}	74	**118**	93	93	50
$\mathrm{Re} = 8 \times 10^4$	256^2	1.2×10^{-5}	**80**	79	75	79	46
$k_0 = 20$	256^2	1.6×10^{-4}	**89**	74	74	72	49

(shells within 10 % of the reference). In these decaying runs, before decorrelation, the parameter largely organizes the ordering (fig. 6). Relative to explicit order 6, pseudospectral differentiation resolves 1.7 times as many shells where the cutoff carries 10^{-11} of the peak, and falls steadily to 0.83 at 10^{-4}. The compact scheme stays at 1.3–1.6 times up to about 5×10^{-6} and then drops to parity and below. The 2/3 rule falls monotonically from 1.13 to 0.55. 3/2 padding follows undealiased spectral within 8 shells in every case (the open diamonds of fig. 6); it changes the leader in one case ($k_0 = 5$ on 256^2, 109 against compact's 105). Pseudospectral leads clearly below about 10^{-8}, and compact clearly near 5×10^{-6}. Between about 10^{-7} and 2×10^{-6} the two trade places: at 9.0×10^{-8} and 2.3×10^{-7} compact is ahead, at 2.0×10^{-7} and 1.6×10^{-6} pseudospectral. So within that band something besides the energy at the cutoff decides. Above about 10^{-5} neither high-resolution scheme beats explicit order 6, whose underestimated derivatives near the cutoff behave like a numerical damping there. The classical resolution measure does better. With $l_\eta = (\nu^3/\eta)^{1/6}$ the enstrophy-dissipation length of the reference at the comparison time (η the sum of its `D_Z` column, which equals 2ν times its palinstrophy to 0.1 %) and $k_{\max} = \pi/h$, the ten cases have $k_{\max} l_\eta = 0.76$–5.0, and ordered by it the leading scheme changes monotonically: explicit order 6 at 0.76 and 0.92, compact at 1.35 and 1.52, the two within 4 % at 1.83 and 2.01, pseudospectral from 2.50 up. The boundaries lie near the thresholds of Lunasin et al. [11] for a resolved ($k_{\max} l_\eta > 1$) and a well-resolved (≥ 2) two-dimensional simulation, though ten points place them only roughly. The cell Reynolds number $\mathrm{Re}\, h$ does not organize the cases at all: the three 256^2 cases at $\mathrm{Re}\, h = 78$, which differ only in k_0, have $k_{\max} l_\eta = 0.92$, 1.35 and 2.01 and fall in three regimes. `tools/compare_schemes.py crossover` computes both and plots the three axes (with `:RE` appended to each case).

The cost at equal resolved range, relative to explicit order 6 as in section 8.9, follows the same axis. Compact costs 0.42–0.78 times wherever $E(K_{\mathrm{Nyq}})/E_{\max} \lesssim 5 \times 10^{-6}$. Pseudospectral costs 0.54–0.76 times below about 10^{-8}, and 0.64–1.4 times in the crossover band and above it. In the heavily under-resolved cases ($\gtrsim 10^{-5}$) both cost more (compact 1.9–3.1, pseudospectral 3.4–4.9). The 2/3 rule never costs less (1.3–11), and neither does 3/2 padding: at the same range as undealiased spectral and 1.85 times its step, it costs 1.00–1.31 times explicit order 6 where pseudospectral leads, and 2–9 times in the under-resolved cases.

In forced turbulence the runs decorrelate, so the comparison is between time-mean statistics, and those carry the sampling scatter of the random forcing. With 28 forced modes, the realized enstrophy input over the averaging windows used here varies by about ±8 %. This sets a floor below which the schemes cannot be told apart. The forced direct cascade with hyperviscosity (section 14.11, 256^2, $t = 6$–16) gives the same slope with all four (-3.44 to -3.48), and the enstrophy flux leaves its plateau at the same $K = 66$, where the hyperviscosity takes over. The plateau levels (41.9, 43.0, 46.2, 50.1 for order 6, compact, spectral, spectral 2/3) differ as much as the total enstrophy each run removed, so they reflect the realized input, not the cascade. Three are within one standard deviation and the dealiased run within about two. With the dissipation confined near the cutoff, the scheme makes no measurable difference here.

With plain viscosity ($\mathrm{Re} = 10^5$, hypodrag, no hyperviscosity, 256^2, $t = 4$–10) the dissipation

range is resolved by the flow itself, not by the grid. The reference is a 768^2 pseudospectral run, whose energy at its own Nyquist shell is 1.4×10^{-11} of the peak; at the 256^2 cutoff the energy is 2.8×10^{-7} of the peak, near the decaying crossover. Normalising each spectrum by its own enstrophy flux, $E \propto \Pi_Z^{2/3}$, takes out the realized input. The spectra then agree with the reference to 10 % from $K = 8$ up to $K = 100$ (order 6), 89 (spectral), 84 (compact) and 42 (spectral, 2/3). At this cutoff energy the decaying cases put pseudospectral or compact ahead; here explicit order 6 holds the longest. The differences are within the remaining scatter, though: away from the cutoff the normalised spectra still differ from the reference by up to 8 %. What does stand out is how each scheme behaves at the cutoff. At $K = 120$, order 6 is low (0.64 of the reference) and pseudospectral piles energy up (1.53, and 2.0 at the Nyquist shell). Compact is in between (1.00, overshooting by up to 17 % just below), and the 2/3 rule truncates. So the energy at the cutoff is a useful organizing parameter for transient decaying turbulence, not a universal predictor of how the schemes rank. In these forced cases the schemes are not distinguishable in the inertial range, and they differ at the cutoff in the characteristic ways seen throughout: explicit stencils fall below the reference, pseudospectral piles up, compact lies between.

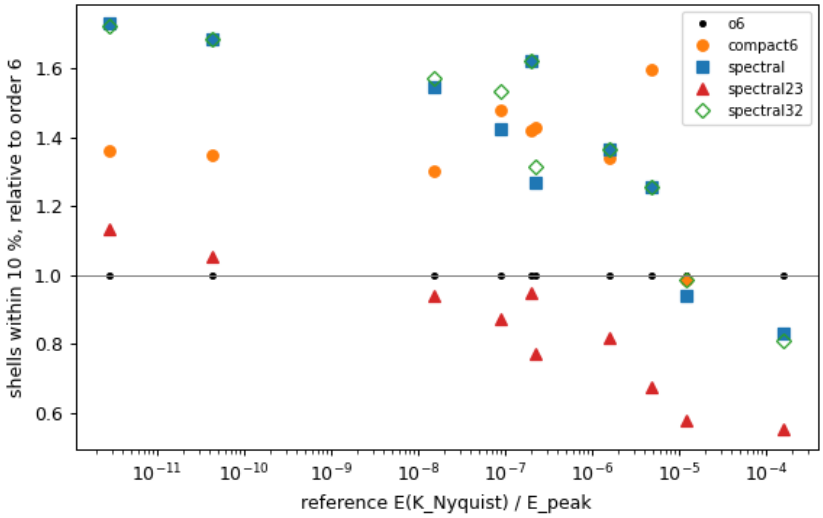


Figure 6: Shells within 10 % of the reference, relative to explicit order 6, against the reference's energy at the grid's Nyquist shell relative to its peak: five decaying cases (Re and k_0 varied about the baseline) on 256^2 and 512^2. Open diamonds: pseudospectral with 3/2 padding, on top of the undealiased squares.

Compact schemes in forced turbulence. The direct-cascade run on 256^2, with the hyperviscosity scaled to that grid ($\nu_h = 1.66 \times 10^{-21}$, $p = 4$), the same seed and time step and averaged over $t = 6$–16, gives nearly the same statistics with both schemes (fig. 7). The enstrophy-flux plateau over $K = 8$–50 is 41.9 (spread 5 %) with order 6 and 42.9 (3 %) compact, and the spectral slope there is -3.44 and -3.48. The flux falls below 90 % of the plateau at $K = 66$ in both, where the hyperviscosity takes over. Over $K = 20$–60 the two spectra agree to 1–2 %; above $K \approx 80$ the compact run carries 8–25 % more energy. That is where the first derivative of order 6 is off by more than 10 % ($K/K_{\mathrm{Nyq}} > 0.54$). With the dissipation confined to that range, the scheme changes the dissipation range and not the inertial one, at 15 times the cost per step in its banded form (66 against 4.4 ms), about 1.4 times in Fourier space. The compact scheme pays off where a flow needs accurate derivatives close to the grid cutoff, as in the decaying runs above, and not in this set-up.

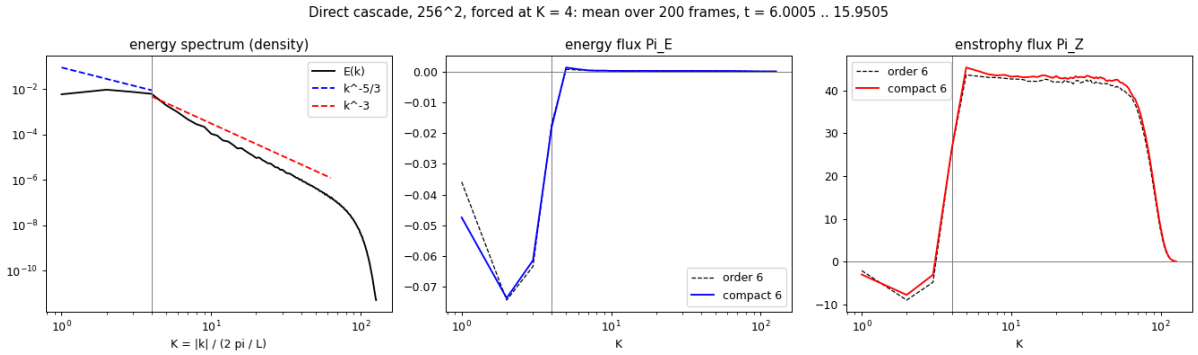


Figure 7: Direct cascade on 256^2 with the compact scheme: time-mean spectrum and fluxes; dashed, explicit order 6.

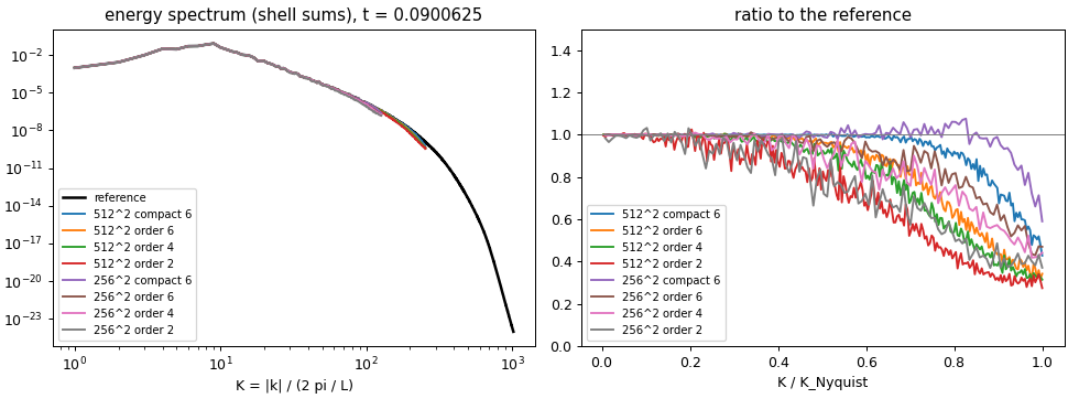


Figure 8: Decaying turbulence at $t = 0.09$: energy spectra of 256^2 and 512^2 runs with orders 2, 4 and 6 and the compact scheme, and their ratio to a 2048^2 order-6 run.

14.12 Physical validation against Ghia et al.

The code embeds the standard Re = 100 centerline velocity values from Ghia et al. [1] and writes them alongside the simulation profile. This supports a conventional cavity-flow validation workflow:

1. run to a demonstrably steady state;
2. interpolate the numerical centerline velocity to the Ghia sample coordinates, or vice versa;
3. compute L_2, L_∞, and selected pointwise errors;
4. repeat on successively refined grids;
5. repeat with smaller timesteps to separate spatial and temporal error.

For the default case (Re = 100, RK4, FFT solver, $t_f = 30$), the simulated centerline velocities interpolated to the 15 interior Ghia points give:

	u on $x = 0.5$		v on $y = 0.5$	
Grid	max error	rms error	max error	rms error
64×64, mixed convention (before)	0.0594	0.0372	0.0595	0.0301
64×64, nodal convention	0.0023	0.0014	0.0073	0.0039
65×65, nodal convention	0.0026	0.0014	0.0076	0.0039
129×129, nodal convention	0.0045	0.0024	0.0089	0.0049
257×257, nodal convention	0.0050	0.0025	0.0091	0.0051

The solution has converged at these resolutions: at the Ghia points the 129×129 and 257×257 profiles differ by at most 4.5×10^{-4} (u) and 3.5×10^{-4} (v), an order of magnitude less than either differs from the reference. The remaining discrepancy is therefore not grid error of this code; it is of the order of the accuracy of the reference data themselves, which were computed on a 129×129 grid with a second-order scheme. The errors grow slightly from 64×64 to 129×129: the smaller error of the coarse grid comes partly from its discretisation error cancelling against the difference to the reference, so 64×64 should not be read as the most accurate grid. The 257×257 run used $\Delta t = 3 \times 10^{-4}$.

`make regression` automates this comparison for the default case: the steady centerline profiles must be within 0.004 (u) and 0.010 (v) of the reference values, which are stored with the tests rather than taken from the program's output, and any value that is not a finite number fails. Grid-convergence rates are not measured on the cavity, for the reason given in section 14.5, but on a manufactured solution.

15 Performance Characteristics

The default FFT-based Poisson solver generally dominates the timestep cost, especially under RK4, which solves four times per step: the first stage reuses the solve that ended the previous step when the interior of ω has not changed since (FFT solvers only; results bitwise the same). That reuse is 8–12 % faster on the cavity, on both backends, and 2–3 % on unforced periodic runs on the GPU; the random kick changes ω every step, so forced runs still do five solves.

Time per step, RK4 + FFT, Re = 100, $\Delta t = 10/n^2$, VTK output off, on an Intel i7-12650H and an NVIDIA GeForce RTX 3050 Laptop GPU (4 GB) under WSL2 (Ubuntu 22.04, gcc 11.4, CUDA 11.5), all builds at the Makefile's `-O2`:

Grid	CPU, `make`	OpenMP, 8 threads	CUDA	CUDA vs CPU
65×65	0.85 ms	0.44 ms	0.94 ms	0.9×
129×129	3.66 ms	1.46 ms	1.05 ms	3.5×
257×257	17.3 ms	6.26 ms	3.62 ms	4.8×
513×513	85.9 ms	40.9 ms	14.8 ms	5.8×
1025×1025	326 ms	171 ms	59.6 ms	5.5×

These are single runs on a laptop; repeat runs usually vary by 10–15 %, occasionally more. They are implementation benchmarks on one machine rather than performance guarantees. Each row is a run such as `./cnavier -n 513 -dt 3.80e-5 -tf 7.62e-3 -output-interval 0`. The GPU pays off from roughly 129×129 upwards; on small grids the fixed cost of launching kernels dominates, and the CPU, especially with OpenMP, is faster. The table is for the FFT solver only. Gauss–Seidel and SOR are on the GPU so that every solver option works there, not because they are fast: on the default 64×64 case SOR takes about 30 ms per step on the GPU, against about 1 ms for the FFT solver.

OpenMP threads, same machine (WSL2 presents it as 8 cores with 2 hardware threads each), RK4 + FFT, best of 2–3 runs:

Threads	129×129	1025×1025
serial build	3.66 ms	326 ms
1	3.55 ms	330 ms
2	2.06 ms	213 ms
4	1.57 ms	179 ms
8 (default here)	1.55 ms	157 ms
16	2.16 ms	191 ms

The 8- and 16-thread rows come from one interleaved comparison (section 12.2 explains the plateau). Forced turbulence on 512^2 takes 16.2 ms per step on the GPU (section 12.3).

For transform performance, grid sizes for which $n-1$ factors into small primes are advantageous, because the sine transform of the $n-2$ interior nodes acts through FFT lengths of $2(n-1)$. Sizes such as 65, 129, 257, 513 and 1025 are therefore fast, and their neighbours can be several times slower: on the machine above one serial FFT Poisson solve takes 6.7 ms at 255^2 ($2(n-1) = 508 = 4 \cdot 127$) against 1.6 ms at 257^2 (512), and 85 ms at 1023^2 against 31 ms at 1025^2. On the GPU a step takes 77 ms at 1024×1024 against 60 ms at 1025×1025.

The GPU becomes increasingly advantageous as the grid grows because the fixed kernel-launch overhead is amortized over more work. Double-precision FFT throughput is a major cost on consumer GPUs, so larger speedups may be observed on hardware with stronger FP64 capability.

16 Building and Running

16.1 Dependencies

The CPU solver requires a C compiler, the math library, and FFTW3. OpenMP is optional. CUDA requires the NVIDIA CUDA toolkit and cuFFT.

On Ubuntu/Debian, a typical CPU setup is

```
sudo apt install build-essential libfftw3-dev
```

On Arch Linux `pacman -S fftw`, and on macOS `brew install fftw`. OpenMP and CUDA are described in sections 12.2 and 12.3.

16.2 Build modes

```
make                    # serial CPU
make OPENMP=1           # OpenMP CPU
make CUDA=1             # CUDA-enabled binary
make OPENMP=1 CUDA=1    # CUDA plus OpenMP CPU fallback/tests
```

The Makefile records the active compiler flags so that changing OpenMP/CUDA modes triggers the required rebuild automatically.

16.3 Tests

```
make test
make CUDA=1 test
make OPENMP=1 CUDA=1 test
make OPENMP=1 convergence   # spatial convergence study, about 1.5 minutes
make OPENMP=1 ablation      # what limits the spatial order
```

`make test` builds and runs `test_cnavier`, which checks:

- **Building blocks**: sparse operations against dense ones; every row of the finite-difference operators (boundary rows included) exact on the polynomials its stencil is built for, for orders 2, 4 and 6; the derivative operators acting along the right axis on non-square grids; wall velocities, corners included; the VTK and centerline writers.
- **Poisson solvers**: the FFT solver against an exact eigenmode, at sizes that exercise every batching case of the transform; FFT, SOR and Gauss–Seidel giving the same answer once converged; the residuals of the iterative solvers; SOR iteration counts on anisotropic grids (section 14.1).
- **Time stepping**: short cavity runs with both schemes (finite, divergence-free); the stability limit (0.95 times runs, 1.05 times diverges) and the suggested Δt; the observed order in Δt, walls included (Euler first, RK4 fourth), also with a time-dependent vorticity source and from $t_0 > 0$.
- **Spatial accuracy**: the manufactured solutions of section 14.5 with walls and of section 14.8 without, at the orders measured there; the compact Poisson operator, the third-order wall closure and the fourth-order velocity rows (section 14.7).
- **Diagnostics, forcing and turbulence terms**: section 14.9 and the sections that follow it.
- **Backends**: driving the solver through `backend.c` gives exactly what calling it directly gives; every solver ignores changes to the caller's configuration after it is created; `backend_psi()` changes no step.
- **Polar grid** (`tests/test_polar.c`): section 21.
- **OpenMP** (with `OPENMP=1`): results on 1 and on 4 threads bitwise identical above the size where loops go parallel.

`make CUDA=1 test` also checks the GPU backend against the CPU: SpMV for all four operators, each Poisson solver, the continuity diagnostic, and full timesteps for the Euler/RK4 and FFT/SOR/Gauss–Seidel combinations on square and non-square grids, with and without a vorticity source, on periodic grids, with the compact Poisson operator and the wall closure from ψ, with the forcing and damping terms, and the spectra. `make OPENMP=1 CUDA=1 test` adds the comparison of the iterative solvers at the default tolerance, meaningful only when the CPU

sweeps in the GPU's red–black order. The test program returns 0 when all executed checks pass, 1 when any check fails, and 77 when a CUDA test build cannot execute its GPU checks because no usable device is present; such a run does not count as a pass.

Further checks, each a `make` target run by continuous integration:

- `test-cli`: invalid command lines (NaN, garbage after numbers, grids out of range, unstable Δt, options that need `-geometry polar` or that it refuses, ...) are refused with exit status 1 and an error message, and write nothing; a crash counts as a failure.
- `regression`: 100 steps of the default case against stored results; the steady default case within 0.004 (u) and 0.010 (v) of the Ghia et al. data (section 14.12); the periodic Taylor–Green vortex against its exact solution on 32^2 and 64^2, at sixth order, and its energy; laminar Kolmogorov flow; the dual cascade of forced turbulence on 64^2 (flux signs and enstrophy conservation, with `tools/cascade.py`); the cylinder at $\mathrm{Re} = 40$ on a coarse polar grid (section 21); and the steady-state check on a short final interval. Non-finite values fail.
- `test-asan` and `valgrind`: the test suite and a short run under AddressSanitizer and UndefinedBehaviorSanitizer, and under valgrind, where any leak, including memory still reachable at exit, fails.
- `format-check`: the formatting of `.clang-format`, which `make format` applies.
- `cppcheck` and `tidy`: static analysis (the checks of `.clang-tidy`), also over the CUDA and OpenMP branches of the C sources; the CUDA source itself is checked only by its compiler.
- `WERROR=1` builds, which make the compiler's warnings errors, nvcc's too.

GitHub Actions (`.github/workflows/ci.yml`) runs, in order: formatting; cppcheck, clang-tidy and `-Wextra -Werror` builds with gcc and clang; serial, OpenMP and CUDA builds; then the test suite, the command-line and regression tests for the serial and OpenMP builds, and the sanitizer and valgrind checks. The CUDA build's test program must report its GPU tests as skipped (exit status 77). The hosted runners have no GPU, so the CPU–GPU comparisons run only where `make CUDA=1 test` is run on a GPU host; the pull request template asks for that whenever the numerics change.

The stored results in `tests/reference/` are deterministic, so `make regression` fails only when the numerics change. If they change on purpose, regenerate them:

```
mkdir -p /tmp/ref/output && (cd /tmp/ref && $OLDPWD/cnavier --tf 0.5 --output-interval
    0)
cp /tmp/ref/output/centerline_u_sim.csv tests/reference/centerline_u_short.csv
cp /tmp/ref/output/centerline_v_sim.csv tests/reference/centerline_v_short.csv
```

and for the cylinder, from the last line of its history:

```
mkdir -p /tmp/cyl/output && (cd /tmp/cyl && $OLDPWD/cnavier --geometry polar --ntheta
    96 --r-inf 20 --diag-interval 500)
python3 -c "import csv; r = list(csv.DictReader(open('/tmp/cyl/output/cylinder_history.
    csv')))[-1]; print('quantity,value'); [print(q + ',' + r[q]) for q in ('cd', '
    wake_length', 'separation_angle', 'E', 'Z', 'P')]" > tests/reference/
    cylinder_regression.csv
```

16.4 Runtime options

Options not given keep the defaults of `src/main.c` (table 3); `./cnavier -help` prints the same list. Values that cannot be used (not a number, a grid outside 8–16384, $t_f/\Delta t$ below one step or beyond the integer range, an unstable Δt) are refused with an error before anything is computed

or written; for a Δt above the limits of section 10 the message gives the limits and a Δt that passes.

Option	Description
`-nx N, -ny N`	Number of grid points in x and y.
`-n N`	Same number of grid points in x and y.
`-dt DT`	Timestep.
`-tf TF`	Final simulation time.
`-output-interval N`	Write VTK every N steps; zero disables VTK output.
`-re RE`	Reynolds number.
`-order N`	Finite differences of order 2, 4 or 6 (default 6), or on the periodic cases `compact6` (section 8.8) or `spectral` (section 8.9).
`-operators NAME`	Periodic cases: apply the derivatives as `sparse` products or in `fourier` space (default `fourier` for `compact6` and `spectral`).
`-dealias, -pad`	With Fourier operators: the 2/3 rule, or dealiasing of the nonlinear term by 3/2 padding.
`-poisson-order N`	2 (five-point, default) or 4 (compact, section 8.6); FFT solver with walls, and the ξ operator of the polar solver.
`-wall-closure NAME`	`velocity` (default on the Cartesian grid) or `briley` (eq. (20); default on the polar grid).
`-velocity-order N`	2 (default) or 4: fourth-order rows next to the walls for u, v (section 14.7).
`-integrals-interval N`	Write E, Z, P, I to `output/integrals.csv` every N steps (default 1; 0 never; section 13.4).
`-drag A`	Periodic cases: linear drag $-\alpha\omega$ (section 8.5).
`-kolmogorov-amp A`	Periodic cases: Kolmogorov forcing amplitude; `-kolmogorov-n` its wavenumber.
`-forcing-rate EPS`	Periodic cases: random forcing rate; `-forcing-k`, `-forcing-width` its shell, `-seed` its seed.
`-hyperviscosity NU`	Periodic cases: hyperviscosity ν_h; `-hyper-order` its order p (default 4).
`-hypodrag A`	Periodic cases: hypodrag α_h (section 8.5).
`-advection NAME`	Periodic cases: `advective` or `skew` nonlinear term (eq. (23)); default `skew` for `forced` and `decaying`.
`-spectrum-interval N`	Periodic cases: write the spectra every N steps (default: with the VTK frames).
`-peak-k K0`	`decaying`: peak of the initial spectrum.
`-case NAME`	`cavity` (default), or on a doubly periodic unit square `taylor-green`, `shear-layer`, `kolmogorov`, `forced` or `decaying` (sections 8.5 and 8.7); with `-geometry polar`, `cylinder`.
`-geometry NAME`	`cartesian` (default) or `polar`: the flow past a cylinder (section 21), with $\mathrm{Re} = 40$, $t_f = 300$, an automatic Δt and the final state only as VTK by default. The options below need it.
`-ntheta N, -nr N`	Polar: grid points in θ (default 256) and r (default: square cells).
`-r-inf R`	Polar: outer radius in diameters (default 40).
`-outer-bc NAME`	Polar: `farfield` (default) or `freestream`.
`-steady-tol TOL`	Polar: stop once $\max \lvert\partial\omega/\partial t\rvert < \mathrm{TOL}$ (default 10^{-6}; 0 never).

Option	Description
`-diag-interval N`	Polar: steps between diagnostics (default 100).
`-cpu`	In a CUDA build, force the CPU backend.
`-help`	Print command-line help.

Example:

```
mkdir -p output
./cnavier --n 127 --dt 6.2e-4 --tf 20 --output-interval 50
```

The time scheme and the Poisson method are configured in `src/main.c`, not on the command line.

17 Default Numerical Configuration

At the repository state documented here, the source defaults are summarized in table 3.

Table 3: Principal defaults in `src/main.c`.

Quantity	Default	Meaning
Re	100	Reynolds number
L_x, L_y	1, 1	domain lengths
n_x, n_y	64, 64	computational array size
Δt	0.005	timestep
t_f	30	final time
finite-difference `order`	6	sixth-order interior stencil
`time_scheme`	2	classical RK4
`poisson_type`	3	FFT/DST-I direct discrete solver
Poisson iterate tolerance	10^{-3}	GS/SOR iterate-change threshold
Poisson max iterations	10000	GS/SOR iteration cap
VTK output interval	20	steps between output files

18 Numerical Interpretation and Current Limitations

The following points are important when interpreting results scientifically.

18.1 Mixed formal spatial order

The transport derivatives may be sixth-order in the deep interior, but the boundary closures are lower order and the Poisson solve is based on a second-order discrete Laplacian. Measured against a smooth manufactured solution with stationary no-slip walls, the complete algorithm converges at second order in space for every derivative order, and orders 4 and 6 give the same errors (section 14.5). This is evidence for one solution family, not a proof for every flow. The ablation of section 14.6 shows that the Poisson operator and the wall-vorticity closure each limit the order to two on their own, so higher-order results need both improved; with the compact Poisson operator and the third-order wall closure (section 14.7) the method reaches fourth order for ω and ψ, and with fourth-order velocity rows next to the walls for the velocity too. Beyond fourth order, the transport operators' rows next to the walls would have to be raised as well. On a doubly periodic grid, with no walls and a Poisson operator of the same order as the derivatives, the method reaches the nominal order 2, 4 or 6 (section 14.8).

18.2 Poisson operator differs from transport Laplacian

The diffusion term in eq. (4) uses the selected $D_{xx} + D_{yy}$ operator. With walls, the streamfunction equation uses by default the standard five-point second-order Laplacian, and with `-poisson-order 4` the compact fourth-order operator (section 8.6); on the polar grid the same choice applies to the ξ operator, and in θ the Poisson operator is $D_{\theta\theta}$ itself (section 21). Only on the doubly periodic grid is the Poisson operator always $D_{xx} + D_{yy}$, of the order of the derivatives (section 8.7). With walls this is a legitimate mixed discretization, but it should be described explicitly because the `order` setting does not uniformly change every spatial operator in the solver: with `-order 6` the Poisson operator is of order 2 or 4.

18.3 Boundary vorticity accuracy

By default on the Cartesian grid, the wall vorticity is reconstructed from differentiated wall velocities, so its accuracy is tied to the one-sided derivative closure and the array-edge interpretation; with the default Poisson operator it limits the method to second order (section 14.6). `-wall-closure briley` uses instead the third-order formula from the streamfunction (eq. (20)), which with the compact Poisson operator makes the method fourth order for ω and ψ (section 14.7); it is the default on the polar grid.

18.4 Explicit stability

At startup a lid-speed Courant check and the viscous bound of the selected time integrator (computed from the assembled second-derivative operator, so it follows the grid, Reynolds number and finite-difference order) are enforced. These are stability bounds only; they say nothing about temporal accuracy, and users remain responsible for timestep-convergence studies.

18.5 Steady-state convergence

The Cartesian driver advances to a specified final time; it does not terminate based on a physical steady-state residual such as $\|\omega^{n+1} - \omega^n\|$ or a momentum residual. (The cylinder run does: it stops once $\max|\partial\omega/\partial t|$ falls below `-steady-tol`, section 21.) For steady cavity studies, t_f should be selected after checking that monitored fields and centerline profiles no longer change materially.

18.6 Pressure is not solved

Pressure is absent from the vorticity–streamfunction formulation. This is an advantage for the cavity velocity field but means that pressure output is not currently a native solved quantity. If pressure is required, it must be reconstructed from an additional pressure-Poisson or momentum-based relation with suitable boundary conditions.

18.7 Geometry and dimensionality

The current method is two-dimensional, on a Cartesian grid or, for the flow past a circular cylinder, a log-polar one (section 21). General curvilinear grids, immersed boundaries, unstructured meshes, three-dimensional vorticity stretching, and general inflow/outflow boundaries are outside the present scope.

19 Suggested Scientific Workflow

For a reproducible numerical experiment, the following workflow is recommended:

1. Record the Git commit, compiler, optimization flags, FFTW/CUDA versions, CPU/GPU, and runtime backend.
2. Run `make test` for the chosen CPU build; for CUDA work, also run the GPU comparison suite on the actual GPU host.
3. Select a baseline grid and timestep conservatively.
4. Confirm temporal convergence by reducing Δt while holding the grid fixed.
5. Confirm spatial convergence with at least three grids while keeping temporal error smaller than spatial error.
6. For the lid-driven cavity, compare centerline profiles against Ghia et al. and report interpolation/error norms.
7. For steady solutions, verify that extending t_f does not materially change the solution.
8. Report the discrete divergence extrema as an implementation consistency diagnostic, but do not substitute them for solution-convergence metrics.

20 Possible Methodological Extensions

The current architecture makes several research-oriented extensions relatively natural:

- a consistent fourth- or sixth-order Poisson discretization, solved by an appropriate sparse/direct/iterative method;
- dedicated high-order wall-vorticity formulas;
- adaptive timestep selection from advective and diffusive spectral bounds;
- automated steady-state residual monitoring;
- automated Ghia interpolation and error norms in the test suite;
- grid-convergence-index (GCI) reporting;
- alternative convection schemes, such as upwind-biased, compact or WENO, and with walls a skew-symmetric form (on periodic grids it is available, eq. (23));
- multigrid for the Poisson equation when boundary conditions or geometries no longer permit a separable sine transform;
- pressure reconstruction as a post-processing step;

21 Polar Grid: Flow Past a Circular Cylinder

With `-geometry polar` (`solver_config.polar` set; the default, `NULL`, is the Cartesian solver of the previous sections, unchanged) the same vorticity–streamfunction solver runs on a log-polar grid around a circular cylinder of diameter $D = 1$ (radius $R = \frac{1}{2}$) in a uniform stream $U = 1$, with $\mathrm{Re} = UD/\nu$.

21.1 Conformal coordinates

The radius is mapped as $r = R\,e^{\xi}$, with $\xi \in [0, \xi_\infty]$, $\xi_\infty = \ln(r_\infty/R)$, and $\theta \in [0, 2\pi)$, both uniform. The map is conformal with scale factor $h = r$, so

$$\psi_{\xi\xi} + \psi_{\theta\theta} = -r^2\omega, \tag{65}$$

$$u_r = \frac{1}{r}\,\psi_\theta, \qquad u_\theta = -\frac{1}{r}\,\psi_\xi, \tag{66}$$

$$\frac{\partial\omega}{\partial t} = -\frac{u_\theta\,\omega_\theta + u_r\,\omega_\xi}{r} + \frac{\omega_{\xi\xi} + \omega_{\theta\theta}}{\mathrm{Re}\,r^2}. \tag{67}$$

In the computational plane these are the Cartesian equations with one factor per row: $1/r$ on the advection and $1/r^2$ on the Laplacian. The fields keep the solver's layout: column j is $\theta_j = j\,\Delta\theta$, $\Delta\theta = 2\pi/n_\theta$ (periodic, built with `SDiff1_periodic` and `SDiff2_periodic`); row i is $\xi_i = i\,\Delta\xi$ (built with `SDiff1` and `SDiff2` and their one-sided end rows), with the cylinder on row 0. The solver's u is u_θ and its v is u_r. The pair (θ, ξ) is left-handed where (r, θ) is right-handed, so the velocity is $u = -D_y\psi/r$, $v = D_x\psi/r$, with the opposite signs of the Cartesian $u = D_y\psi$, $v = -D_x\psi$. The divergence $(D_y(rv) + D_x(ru))/r^2 = (D_yD_x - D_xD_y)\psi/r^2$ vanishes to round-off, because the Kronecker operators commute. By default the cells are square, $\Delta\xi \approx \Delta\theta$, so the cell size grows linearly with r, and an outer radius of $40D$ costs $n_r \approx 0.7\,n_\theta$ points. $\theta = 0$ points downstream. The cosine and sine tables are mirror-symmetric to the last bit, with $\cos\theta = 0$ exactly at $\theta = \pi/2$ and $3\pi/2$, so the scheme preserves the symmetry of the flow about the wake axis to round-off.

21.2 Poisson solver

A real FFT in θ gives, for each mode m,

$$\delta_{\xi\xi}\hat\psi_m + s_m\hat\psi_m = \widehat{(-r^2\omega)}_m, \qquad \hat\psi_m(0) = 0, \quad \hat\psi_m(\xi_\infty) = \widehat{(\psi_{\mathrm{out}})}_m, \tag{68}$$

where s_m is the symbol of $D_{\theta\theta}$ (read from the operator's first row, as for the periodic solver), so that the θ part of the Poisson operator matches the transport's. In ξ the operator is the three-point one (`-poisson-order 2`) or the compact fourth-order one, $\delta^2\hat\psi + s_m(1 + \Delta\xi^2\delta^2/12)\hat\psi = (1 + \Delta\xi^2\delta^2/12)\hat f$ (`-poisson-order 4`), which is still tridiagonal. Its right-hand side on the boundary rows is extrapolated (cubically) from the interior. Each mode is solved by the Thomas algorithm with factors computed once. The systems are diagonally dominant for every $s_m \le 0$.

21.3 Boundary conditions

Cylinder. No slip: $\psi = 0$, $u = v = 0$ on row 0. The wall vorticity is $\omega_w = -\psi_{\xi\xi}/R^2$, from Briley's third-order formula in ξ (`-wall-closure briley`, the default here), or $\omega_w = (D_y(ru) - D_x(rv))/r^2$ with the one-sided rows (`velocity`).

Outer boundary. The Poisson solver needs one boundary type for every θ, so ψ is Dirichlet, $\psi = \psi_{\mathrm{out}}(\theta)$. On the inflow part ($\cos\theta < 0$) $\omega = 0$, and on the outflow part it has zero normal gradient. With `-outer-bc freestream`, $\psi_{\mathrm{out}} = r_\infty \sin\theta$. With `farfield` (the default), following Gautier, Biau and Lamballais [12] the far velocity is $\boldsymbol{u}_\infty + \boldsymbol{u}_1 + \boldsymbol{u}_2$. $\boldsymbol{u}_1$ is the Schlichting wake, with deficit

$$d = \frac{C_D\sqrt{\mathrm{Re}}}{4\sqrt{\pi}}\,\frac{e^{-\mathrm{Re}\,r(1-\cos\theta)/2}}{\sqrt{r}} \tag{69}$$

in u_x and $u_y = -\frac{1}{2}\sin\theta\,d$. $\boldsymbol{u}_2$ is a uniform radial outflow that returns the flux the deficit removes (Hasan et al.), which is what makes $\psi_{\mathrm{out}} = r_\infty\sin\theta + \int_0^\theta r_\infty(\boldsymbol{u}_1 + \boldsymbol{u}_2)\cdot\boldsymbol{e}_r\,\mathrm{d}\theta'$ single-valued. The integral is done by FFT. The condition carries the drag coefficient of the body: the run follows its own C_D, under-relaxed by $\frac{1}{2}$ at every diagnostic step from $t = 5$ on, and the boundary values change only between steps.

21.4 Time step and diagnostics

The viscous term is stiffest on the cylinder, so the limits are those of the uniform (θ, ξ) grid at Reynolds number $\mathrm{Re}R^2$, and the Courant limit uses the smallest cell $R\min(\Delta\xi, \Delta\theta)$ and the speed $2U$, the peak of potential flow. Without `-dt` the step is 0.9 of the smaller limit. The run stops at steady state, when $\max|\partial\omega/\partial t|$ over the interior falls below `-steady-tol`.

- **Forces** from the wall vorticity alone. The friction is $\tau = \nu\omega_w$. On a no-slip wall the θ-momentum equation gives $\partial p/\partial\theta = R\nu\,\partial\omega/\partial r = \nu\,\omega_\xi$, so integrating by parts, $C_{D,p} = \nu \oint \omega_\xi \sin\theta\,\mathrm{d}\theta$ and $C_{D,f} = -\nu \oint \omega_w \sin\theta\,\mathrm{d}\theta$ (trapezoidal, spectrally accurate for these periodic integrands). The wall pressure, integrated by FFT, is written to `output/cylinder_surface.csv`.
- **Separation angle** θ_s (from the front stagnation point): the zero of ω_w on the upper surface, by cubic interpolation. **Wake length** L_w/D: the zero of u_r on the wake axis, less R. **Recirculation centre** (a, b): the minimum of ψ in the upper bubble, refined by parabolas; a downstream of the rear stagnation point, b the distance between the two centres.
- The velocity at any point by cubic Lagrange interpolation in (ξ, θ), for the reference value at $(x, y) = (3, 3)$; the asymmetry $\max|\omega(\theta) + \omega(-\theta)|$; and the continuity check.

The GPU backend is Cartesian only. A polar configuration runs on the CPU, with a message, even when the GPU is asked for. `rk4_alloc()` refuses (`polar_config_error()`) every option the polar right-hand side does not implement: the forcing and damping terms, the Fourier operators, the skew-symmetric form, the fourth-order velocity rows and the four walls of `wall_bc`; the cylinder's velocity is `polar_grid.wall_ut`. The outer boundary values may change between steps through the setters of `polar.h` only, which bump a version that tells the solver (and `backend_psi()`) not to reuse a stream function solved for the old values.

21.5 Running the case

```
./cnavier --geometry polar --ntheta 128     # 128 x 90, r_inf = 40: about a minute on
    one core
```

The options are listed in section 16.4. At the end the run prints its comparison with the reference values (`tests/reference/cylinder_re40.csv`), and it writes:

- `output/cylinder_history.csv`: C_D with its pressure and friction parts, L_w/D, θ_s, $\max|\partial\omega/\partial t|$ over the steps since the previous line, the C_D of the far-field condition, and the area means E, Z, P (section 13.4, with $\mathrm{d}A = r^2\,\mathrm{d}\xi\,\mathrm{d}\theta$);
- `output/cylinder_surface.csv`: the wall vorticity and wall pressure against θ;
- `output/cylinder_centreline.csv`: u_x on the symmetry axis, upstream and in the wake;
- VTK structured grids of the vorticity (with the Cartesian velocity) and of the stream function, in Cartesian coordinates and closed at $\theta = 2\pi$.

`tools/cylinder_validate.py` runs series of grids and outer radii (`run`), tabulates them against the reference with observed orders and Richardson extrapolation (`table`), and plots the centreline velocity and the wall vorticity (`plot`).

21.6 Verification and validation

`tests/test_polar.c` checks the following.

- The Poisson solver against potential flow, $\psi = A(r - R^2/r)\sin\theta$: observed orders 2.00 and 3.99. The second-order operator also reproduces its own discrete system to 2×10^{-14}.

- The velocity signs and its divergence (3×10^{-15}).
- A manufactured solution of the full solver, $\psi = g(t)(r - R)^2(r_\infty - r)^3 T(\theta)$, with three θ modes and its source in closed form. It converges at second order with order-2 operators and at about 2.7 with orders 4 and 6, the compact Poisson operator and Briley's closure.
- The diagnostics on fields with known answers; the area means exact on constants and second order against closed forms.
- The symmetry of the impulsive start, and the CPU fallback.
- Every unsupported configuration option refused, one at a time.
- An outer boundary changed between steps reaches the next step: the step equals a fresh solver's, bitwise (it fails if the version check is removed). `backend_psi()` is the Poisson solve of ω before the first step, after `backend_set_fields()` and after a change of the boundary, and calling it changes no step.

`make regression` runs the 96×57, $r_\infty = 20$ case (about 20 s) against its stored results (`tests/reference/cylinder_regression.csv`, 10^{-4}), with a looser sanity check against the reference, and checks that a run whose last diagnostic interval is shorter than the others reports that it has not converged. The validation is against the reference solution of Gautier et al. [12] at $\mathrm{Re} = 40$, a pseudo-spectral computation on 200×1024 points with $r_\infty = 40D$: $C_D = 1.4931$, $L_w/D = 2.2360$, $\theta_s = 126.39°$, $(a, b)/D = (0.71, 0.59)$. Table 4 gives the steady states (order 6, RK4, far-field boundary unless stated), from `tools/cylinder_validate.py`. From 128×90 up,

Table 4: Steady flow past a cylinder at $\mathrm{Re} = 40$ against Gautier et al. [12]: order 6, RK4, Poisson order 2 unless stated; in parentheses the difference from the reference. u_x, u_y: the velocity at $(x, y) = (3, 3)$.

Grid ($n_\theta \times n_r$)	r_∞	Outer	C_D	L_w/D	θ_s (°)	$(a, b)/D$	u_x, u_y
64×46	40	far field	1.4716 (−1.44%)	2.2587 (+1.02%)	126.291	0.81, 0.59	1.0691, 0.0148
128×90	40	far field	1.4903 (−0.19%)	2.2459 (+0.44%)	126.359	0.70, 0.60	1.0714, 0.0150
256×180	40	far field	1.4969 (+0.26%)	2.2431 (+0.32%)	126.356	0.70, 0.59	1.0711, 0.0149
256×180, Poisson 4	40	far field	1.4972 (+0.27%)	2.2421 (+0.27%)	126.354	0.70, 0.59	1.0711, 0.0149
256×168	30	far field	1.5002 (+0.48%)	2.2429 (+0.31%)	126.347	0.70, 0.59	1.0725, 0.0147
256×189	50	far field	1.4953 (+0.15%)	2.2428 (+0.30%)	126.361	0.70, 0.59	1.0705, 0.0150
256×180	40	free stream	1.5134 (+1.36%)	2.2473 (+0.51%)	126.300	0.70, 0.59	1.0780, 0.0140
Gautier et al.	40		1.4931	2.2360	126.395	0.71, 0.59	1.0670, 0.0151

the far-field runs are within 0.5% of the reference in C_D and L_w/D and within 0.05° in θ_s, inside the 10% scatter of the published values ($1.48 < C_D < 1.62$). The remaining difference comes from the outer boundary, not the grid.

- C_D converges at an observed order of 1.5, to 1.5006 by Richardson extrapolation.
- L_w/D is grid-converged with the compact Poisson operator: 2.2420 on 128×90 and 2.2421 on 256×180.
- C_D changes by 0.3% between $r_\infty = 30$ and 50, in the opposite direction to the reference's own trend. The reference extrapolates the velocity from the interior at the outer boundary, where the far-field model here fixes ψ.

The free-stream boundary gives C_D 1.4% high and accelerates the centreline flow spuriously to $u_x = 1$ over the last ten diameters, the "Dirichlet" behaviour of the reference's Figure 1 (fig. 9). The far-field boundary removes most of it: the centreline velocity ends at the Schlichting deficit,

$u_x \approx 0.79$, with a mild upturn in the last few diameters. Every run is symmetric to 2×10^{-14} and divergence-free to 3×10^{-13}. The 128×90 grid takes about a minute on one core; 256×180 takes 110 000 steps at 5.6 ms (6 threads), about 11 minutes.

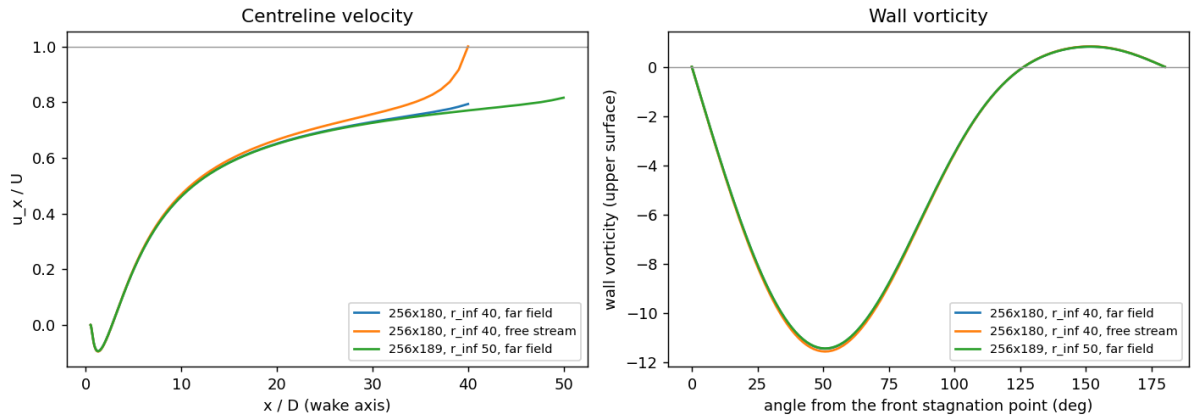


Figure 9: Steady flow past a cylinder at Re = 40, 256×180: the velocity on the wake axis (left) and the wall vorticity on the upper surface (right), with the far-field boundary at $r_\infty = 40$ and 50 and the free-stream boundary at $r_\infty = 40$.

22 Final Remarks

`cnavier` implements a mathematically clean and computationally compact route to incompressible flow in two dimensions: evolve vorticity, invert a scalar Poisson equation, and differentiate a streamfunction to recover velocity. The code is particularly transparent because the spatial derivatives are explicit finite-difference matrices, the two-dimensional operators are tensor products, and both CPU and GPU backends expose the same algorithmic sequence.

The strongest parts of the present methodology are the exact discrete streamfunction enforcement of incompressibility, the direct DST-based Poisson option, reusable sparse operators, RK4 stage consistency, and unusually thorough CPU–GPU equivalence tests for a compact CFD code. The main scientific qualifications are equally clear: high-order accuracy applies primarily to interior transport derivatives; boundary closures are lower order; the streamfunction Poisson operator is second order; and quantitative benchmark/grid-convergence validation is not yet automated.

For educational CFD, numerical experimentation, backend comparisons, and as a foundation for further research-oriented development, these characteristics make the project both useful and unusually auditable. For publication-quality quantitative claims, the next step is not to complicate the code immediately, but to formalize its grid convention and add systematic temporal/spatial convergence and benchmark-error measurements.

A Mapping Between Equations and Source Code

Mathematical operation	Main implementation location
First-derivative stencils	`SDiff1()` in `src/finitediff.c`
Second-derivative stencils	`SDiff2()` in `src/finitediff.c`
Kronecker operators	`skronecker()` in `src/linearalg.c`; assembly in `src/main.c`

Mathematical operation	Main implementation location
CSR derivative evaluation	`spmv()` in `src/linearalg.c`; `spmv_kernel` on CUDA
Wall velocity condition	`apply_wall_bc()` / `wall_bc_kernel`
Wall vorticity	`step()` / `vorticity_bc_kernel`
Vorticity RHS	`dwdt()`, `euler()` / `rhs_kernel`
RK4	`rk4()` / CUDA stage kernels in `gpu_step()`
Gauss–Seidel	`poisson()` / `poisson_redblack()`
SOR	`poisson_SOR()` / `poisson_redblack()`
FFT/DST-I Poisson	`poisson_FFT()` / `poisson_fft()`
Velocity recovery	`velocity_from_vorticity()` / `velocity_kernel`
Divergence diagnostic	main-loop CSR operations / `continuity_kernel`
Ghia centerlines	`print_centerline()` in `src/utils.c`
Verification suite	`tests/test_solver.c`
Polar grid, Poisson solver, cylinder diagnostics	`src/polar.c` (`polar.h`)
Flow past a cylinder	`cylinder_run()` in `src/cylinder.c`
Polar tests	`tests/test_polar.c`

B Reproducibility Record

This document was prepared from the public repository

https://github.com/leo-aa88/cnavier

with `main` observed at commit

`5018e29bff534b252a7ac9025ece4ac43efd78ad`

on 6 October 2026. Numerical details in this manual describe the source code at that repository state; later commits may change defaults or algorithms.

References

[1] U. Ghia, K. N. Ghia, and C. T. Shin, "High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method," *Journal of Computational Physics*, vol. 48, no. 3, pp. 387–411, 1982.

[2] J. H. Ferziger, M. Perić, and R. L. Street, *Computational Methods for Fluid Dynamics*, 4th ed., Springer, 2020.

[3] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.

[4] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.

[5] S. K. Lele, "Compact finite difference schemes with spectral-like resolution," *Journal of Computational Physics*, vol. 103, no. 1, pp. 16–42, 1992.

[6] R. H. Kraichnan, "Inertial ranges in two-dimensional turbulence," *Physics of Fluids*, vol. 10, no. 7, pp. 1417–1423, 1967.

[7] J. C. McWilliams, "The emergence of isolated coherent vortices in turbulent flow," *Journal of Fluid Mechanics*, vol. 146, pp. 21–43, 1984.

[8] G. Boffetta and R. E. Ecke, "Two-dimensional turbulence," *Annual Review of Fluid Mechanics*, vol. 44, pp. 427–451, 2012.

[9] U. Frisch, S. Kurien, R. Pandit, W. Pauls, S. S. Ray, A. Wirth and J.-Z. Zhu, "Hyperviscosity, Galerkin truncation, and bottlenecks in turbulence," *Physical Review Letters*, vol. 101, p. 144501, 2008.

[10] C. Canuto, M. Y. Hussaini, A. Quarteroni and T. A. Zang, *Spectral Methods: Fundamentals in Single Domains*, Springer, 2006.

[11] E. Lunasin, S. Kurien, M. Taylor and E. S. Titi, A study of the Navier–Stokes-α model for two-dimensional turbulence, *J. Turbul.* **8** (2007) N30.

[12] R. Gautier, D. Biau and E. Lamballais, A reference solution of the flow over a circular cylinder at Re = 40, *Computers & Fluids* **75** (2013) 103–111; arXiv:1310.6641.